

TUNABLE NON-LINEAR MANIFOLD ROMs FOR ELIPTIC, PARABOLIC, AND HYPERBOLIC PDES VIA MATRIX-FREE PETROV–GALERKIN PROJECTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Neural operators such as Fourier Neural Operators and DeepONets typically deliver one (accuracy, speed) operating point per trained model; moving it generally requires retraining. We present a nonlinear-manifold reduced-order model (NM-ROM) for Poisson, heat and viscous Burgers equations that exposes a deployment-time accuracy–speed tradeoff from one trained decoder, set by the correction rank q (accuracy) and the solver tolerance, budget and empirical-quadrature rule (cost). The framework combines a frozen coordinate-network spatial bank with a nonlinear head and nested correction directions; exact Dirichlet enforcement in the bank; least-squares Petrov–Galerkin projection of the discrete residual onto fixed weak tests, with every linear operator preassembled; matrix-free JAX evaluation; and NNLS empirical quadrature for Burgers advection, validated on held-out reached states but not confirmed on independent re-draws at 4096^2 . Across Poisson and heat in two and three dimensions, and viscous Burgers in two, at meshes up to 4096^2 , the frozen model keeps its accuracy as the two-dimensional mesh is refined, so its speedup over the named iterative solvers reaches $116\times$ at 0.96 % error against conjugate gradients on Poisson, $36.0\times$ at 0.49 % against Crank–Nicolson CG on a sealed held-out heat cohort, and $4.87\times$ at 0.60 % on Burgers against Newton–BiCGStab ($5.38\times$ at 1.33 % on held-out cases); at matched latent dimension its fast setting is more accurate than a reproduced shallow-masked-autoencoder NM-ROM and POD-LSPG. The named solvers remain more accurate; three-dimensional heat meets its accuracy target but beats CN–CG only with a batched fit at 128^3 , and three-dimensional Burgers, Navier–Stokes and full-state waves miss their targets.

1 INTRODUCTION

Neural operators for partial differential equations (PDEs) (Li et al., 2021; Lu et al., 2021; Kovachki et al., 2023) typically deliver one (accuracy, wall-clock) operating point per trained model, with no deployment-time knob to trade compute for accuracy; moving the point generally requires retraining. Full-order methods (FOMs) solved with iterative solvers such as conjugate gradients (Hestenes and Stiefel, 1952) expose this knob through their tolerance, but their cost grows with the mesh. Reduced-order models (ROMs) approach the problem from the opposite direction: linear projection-based ROMs (Benner et al., 2015; Sirovich, 1987) can require many basis functions for moving fronts (Cohen and DeVore, 2015), and nonlinear-manifold ROMs (NM-ROMs) learn a more compact representation (Lee and Carlberg, 2020; Kim et al., 2022), but their online nonlinear solve can offset the benefit of fewer unknowns. The key open question is thus: *can one trained NM-ROM give a deployment-time accuracy–cost choice, keep its accuracy as the mesh is refined, and be faster than named iterative full-order solvers while being more accurate than prior nonlinear-manifold ROMs?*

This paper answers that question largely affirmatively for two-dimensional Poisson, heat and viscous Burgers. We present an NM-ROM whose distinguishing property is a deployment-time accuracy–cost choice from a single trained model. It represents the solution by a learned spatial bank multiplied by a nonlinear map of latent variables, augmented with nested, precomputed correction directions: activating more directions enlarges the trial space without changing the learned weights. The correction rank is the accuracy control; stopping tolerances and empirical quadrature set the cost of the solve. All online coefficients come from the weak PDE residual, without access to the reference

solution. The accurate error (Table 1) stays near its coarse-mesh level up to 4096^2 while the speedup over the named solver grows, and on held-out Burgers cases the model is far more accurate than a reproduced NM-ROM and POD-LSPG (Table 2). Three-dimensional heat meets its accuracy target but is faster than CN-CG only with a batched fit at 128^3 ; it is not yet so for three-dimensional Burgers, Navier–Stokes or the full wave state (Table 3).

Contributions.

1. **A tunable NM-ROM with a deployment-time accuracy/speed tradeoff from a single trained model.** Nested correction directions provide discrete deployment settings that change approximation capacity without retraining. We isolate correction rank at fixed residual test count and separately evaluate a prescribed test-count schedule across seeds and held-out cases.
2. **Architectural choices that make this tradeoff achievable.** A learned spatial bank, a nonlinear head with a linear skip, and nested corrections support compact latent solves, boundary enforcement and node-local evaluation.
3. **A matrix-free JAX implementation that makes the framework practical.** With precomputed linear operators, matrix-free differentiation and validated empirical quadrature, query time grows more slowly with the mesh than the named solvers’: the accurate setting reaches 0.96 % on Poisson at $116\times$ the speed of CG, and 0.60 % on Burgers at $4.87\times$ that of Newton–BiCGStab, at 4096^2 in the same allocation.

2 RELATED WORK

Linear and non-linear manifold ROMs. Linear ROMs approximate solutions in a fixed subspace (Benner et al., 2015; Sirovich, 1987); moving fronts can require a large basis (Cohen and DeVore, 2015; Greif and Urban, 2019). Nonlinear-manifold ROMs combine learned representations with residual projection (Lee and Carlberg, 2020; Kim et al., 2022), including quadratic manifolds (Geelen et al., 2022; Barnett and Farhat, 2022), neural-field representations (Kim et al., 2024) and adaptive bases (Peherstorfer and Willcox, 2015; Carlberg, 2015). Least-squares Petrov–Galerkin projection (Carlberg et al., 2011) and preassembly of reduced operators (Ştefănescu and Sandu, 2014; Weder et al., 2024) are established components. We differ from these by a coordinate-network bank with an exact boundary factor, a head with a linear skip, and nested corrections selected after training that give deployment-time settings from one trained model, as runtime-tunable networks do (Yu et al., 2019; Cai et al., 2020).

Neural operators and hyper-reduction. FNO (Li et al., 2021), DeepONet (Lu et al., 2021), U-Net (Ronneberger et al., 2015; Takamoto et al., 2022) and Transolver (Wu et al., 2024) learn solution maps. Their evaluation resolution or inference procedure can also change accuracy and cost; our mechanism instead changes the trial space used to solve the PDE. Empirical quadrature (Hernández et al., 2017; Yano and Patera, 2019) reduces nonlinear residual evaluation through non-negative weights. We retain that construction, testing its error on states reached by the solver and through independent re-draws. The main iterative linear-system comparator is conjugate gradient (Hestenes and Stiefel, 1952).

3 METHODOLOGY

To answer this question, we turn a learned manifold into a practical PDE solver through three components. First, a trial manifold whose decoder enforces Dirichlet conditions exactly and carries nested linear correction directions (§3.1). Second, a least-squares Petrov–Galerkin projection of the discrete residual onto fixed weak tests (§3.2). Third, exact preassembly of every linear term, with Empirical Quadrature for the Burgers advection where validated (§3.3); §3.4 gives the architecture. Throughout, $u \in \mathbb{R}^n$ is the full-order state on a uniform grid of N intervals per axis and $A \in \mathbb{R}^{n \times n}$ the discrete negative Laplacian. Four sizes recur: R spatial basis functions in the bank, k nonlinear latent coordinates, M weak tests (the rows of the projected residual) and m quadrature nodes, with $k + q \ll M \ll n$. The formulas illustrate scalar Dirichlet problems; Appendix A gives the per-PDE derivations (periodic vector fields, time integrators) and exit codes, Figure B.1 the data flow.

3.1 TRIAL MANIFOLD WITH EXACT DIRICHLET ENFORCEMENT

A solution is a fixed set of spatial functions with coefficients that a small network produces. The *bank* $G \in \mathbb{R}^{n \times R}$ holds those R functions, one per column, sampled at the mesh nodes; it is built once per mesh, then frozen. The *head* $h_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^R$ is a small network that turns k latent coordinates z into the R coefficients. The decoder is their product, $\mathcal{D}(z) = Gh_\theta(z)$, $k \leq R \ll n$, and there is no encoder. Each bank column is a random-Fourier-feature coordinate network g_ϕ (Tancik et al., 2020) read at the node coordinates and multiplied by a factor that vanishes on the boundary, here for the two-dimensional square,

$$G_{x,:} = \mu(x) g_\phi(x)^\top, \quad \mu(x) = 16 x_1(1 - x_1) x_2(1 - x_2). \quad (1)$$

Because μ is zero on the boundary Γ , every column of the bank is zero there, so every trial state and every derivative $\partial u / \partial z$ vanishes on Γ at any resolution (homogeneous data only); no penalty term is needed. The L-shape uses a distance-based factor, the cube a scaled product factor, and periodic Navier–Stokes none (§3.4).

Nested correction directions. The head reaches only the states its k coordinates produce. To widen that set after training we add q fixed directions in coefficient space, the columns of $C_q \in \mathbb{R}^{R \times q}$, whose coefficients y the solver chooses alongside the latent code,

$$u(z, y) = G(h_\theta(z) + C_q y), \quad z \in \mathbb{R}^k, y \in \mathbb{R}^q, 0 \leq q \leq R. \quad (2)$$

The columns of C_q are computed offline from training data alone; for Burgers they are the principal components of the coefficient errors the head leaves behind. Per-PDE constructions are in Appendix A. They are ordered once, so C_q is the first q columns of $C_{q'}$: raising q adds directions without disturbing those in use, and no setting needs retraining. At $q = 0$ the model is the frozen head, solved in k unknowns; at $q = R$ the head no longer matters and the trial states fill the span of the bank. In between, q buys reach: representable states grow from the head’s image towards that span, and the solve from k to $k + q$ unknowns. Two consequences matter later. The bank’s projection error is a floor that no head and no solver can beat, so nonlinearity in h_θ buys a smaller *solved* dimension, not an escape from the span. And all dependence on the spatial coordinate sits in G , which is what lets us sample nodes and preassemble operators from one decoder.

3.2 PROJECTING THE PDE ONTO THE MANIFOLD

Each PDE gives a discrete residual $r(u) \in \mathbb{R}^n$: the amount by which a candidate state fails the discrete equations, zero at the discrete solution. A trial state has only $k + q$ free numbers, so r cannot be driven to zero; we ask instead that it be small against M fixed directions. The rows of $P \in \mathbb{R}^{M \times n}$ are those *weak tests*: on the Dirichlet square they are the low-frequency tensor-product sine vectors, which are eigenvectors of the five-point operator, $PA = \Lambda P$; the L-shape uses Lanczos eigenvectors of its own operator. Substituting the trial state into r and testing it gives the problem solved at every query,

$$(z^*, y^*) = \arg \min_{z, y} \frac{1}{2} \|r_w(z, y)\|_2^2, \quad r_w(z, y) = \Lambda_\star^{-1} P r(u(z, y)) \in \mathbb{R}^M, \quad M > k + q, \quad (3)$$

The diagonal row scaling $\Lambda_\star$ (per PDE, Appendix A) puts the rows on a common footing. This is least-squares Petrov–Galerkin projection (Carlberg et al., 2011; Lee and Carlberg, 2020): the test space is fixed, not the moving tangent space of Galerkin projection, and $M > k + q$ makes the system overdetermined, not square.

Elliptic (Poisson). The residual is $r(u) = Au - f$, so testing it leaves $r_w(z, y) = B_0(h_\theta(z) + C_q y) - b_0$, where $B_0 = PG$ holds the tested bank functions and b_0 the tested source (Appendix A.1). Because that residual is linear in the coefficients, the corrections cost no extra iterations: the projector onto $\text{range}(B_0 C_q)$ does not depend on z , so y is eliminated exactly (Golub and Pereyra, 1973), the iteration stays k -dimensional at every q , and $q = R$ is a linear solve.

Parabolic (heat). The semi-discretisation $du/dt = -\kappa Au$ is advanced with Crank–Nicolson. The manifold enters the fully discrete step *before* projection, and each step solves $z_{n+1} = \arg \min_z \|B_0 h_\theta(z) - D B_0 h_\theta(z_n)\|_2$, where D scales each test by its Crank–Nicolson amplification

factor (Appendix eq. (6), shown at $q = 0$; with corrections, Appendix A.2). The step is a nonlinear least-squares problem, not a linear solve; at $q = R$ it becomes the linear recurrence of Appendix eq. (8). Reflective waves use an explicit latent integration instead (Appendix A.4).

Hyperbolic (Burgers). For $u_t + u(u_x + u_y) = \nu \Delta u$ with a sign-upwind stencil and backward Euler (Appendix A.3) the tested residual is nonlinear in the coefficients, so y cannot be eliminated in closed form. We damp the z and y blocks separately inside one Levenberg–Marquardt step, leaving the linear block undamped (*block-damped* variable projection). In the recorded solver comparison this removes the joint configuration’s 6 budget exits (Table E.3).

Solver and exits. The nonlinear problems are solved by damped Levenberg–Marquardt with a monotone test (Marquardt, 1963): each attempt solves a damped normal system for a step in the $k + q$ unknowns and keeps it only if the residual strictly decreases. Three exit families are recorded and never conflated: a scale-free stationarity measure, a residual threshold, and the stalls, reported as early-stopped and never as converged. No query uses the solution it predicts; every solve starts from stored training codes (Appendix A.6).

3.3 HYPER-REDUCTION

The manifold cuts the unknowns from n to $k + q$, but a tested term such as $PN(Gc)$ still touches every node, so one residual still costs $O(n)$. Linear terms escape this: they can be tested once, offline, $P(Au - f) = B(h_\theta(z) + C_q y) - b$, where the $M \times R$ matrix $B = \Lambda PG = \Lambda B_0$ is the B_0 of §3.2 with its rows rescaled, assembled offline by the discrete transform of each problem. Linear terms are therefore exact and cost nothing per node online. The Burgers advection is the one term that resists this. It is evaluated *densely*, at every node, at $O(nR)$ per residual, or on an *empirical quadrature* rule (Hernández et al., 2017; Yano and Patera, 2019) that replaces the sum over all nodes by a weighted sum over m of them, at $O(mR)$. The rule is one non-negative weight per node, the same for every state, fitted by NNLS (Lawson and Hanson, 1974) to reproduce the tested advection term at n_{fit} stored codes under a cap of m nodes (Appendix A.5). Changing m re-solves the fit, so rules are not nested; deployment picks among stored rules. *A rule is never accepted on its NNLS fit residual.* We score it on states the solver actually reaches, on trajectories disjoint from the fit and the evaluation cases, against a bar fixed before any certification job ran, and call it *confirmed* only if independent re-draws of those trajectories also pass (Appendix A.5, Table E.2).

3.4 MODEL ARCHITECTURE

Three properties of the problem motivate our architecture. First, every query starts from stored training codes and descends along the head’s Jacobian; this motivates a *linear skip*. Second, quadrature evaluates the decoder only near sparse nodes, and one model serves every mesh; this motivates a *coordinate-network bank*. Third, one model must offer several accuracy–cost settings; this motivates *nested corrections*.

Linear skip, for the latent solve. $h_\theta(z) = \varphi_\theta(z) + W^\top z$, with φ_θ an MLP with two hidden SiLU layers; the skip keeps a latent-independent Jacobian component. Widths and (k, R) are per family (Tables D.2 and D.11); the skip is a design choice, not ablated here.

Coordinate-network bank, for node-local evaluation. The vanishing factor of (1) becomes a product form in 3D and a distance-based factor on the L-shape; some banks are right-multiplied by a fixed orthonormalising matrix, and the periodic Navier–Stokes bank has no factor but a global solenoidal projection. Because g_ϕ takes coordinates, not a grid, one node can be decoded without the others: a quadrature rule’s support is a cached block, and decoding in row blocks at large n changes memory, not the model.

Nested corrections, for accuracy. C_q holds leading singular directions of training coefficient residuals, nested in q .

What is fixed and what is solved. Fixed by training, and shared by every row of Table 1: the bank G , the head weights, the directions C_q and the tests P . Solved at every query: the code z and

the coefficients y , once per time step when time enters. Chosen at run time: q , the test count M (usually $M = 4(k + q)$; fixed along the rank study of Table D.8 and on the L-shape), the quadrature, the tolerance and budget, and for heat the stepping mode. Three error quantities separate the parts (Appendix A.6).

4 IMPLEMENTATION

4.1 MATRIX-FREE PROJECTED OPERATORS VIA JAX

The $M \times R$ matrix B of §3.3 is built once per mesh, so a linear PDE never touches the grid online: its residual is $B(h_\theta(z) + C_q y) - b$ and its Jacobian $B[Dh_\theta \ C_q]$, where Dh_θ is the head’s Jacobian, obtained by forward-mode Jacobian-vector products (Bradbury et al., 2018). The sampled Burgers advection reads only the cached stencil block. The damped normal system is solved directly; no Krylov solve is applied to the projected operator. The Burgers runs above 1024^2 assemble the Jacobian analytically, with the solver refinements of Appendix A.3.

4.2 TRAINING PROTOCOL

Training runs in two stages, both as auto-decoders and both on training data only. First the coordinate network g_ϕ is fitted to the training states through (1) and frozen, which fixes the bank. Then the head is fitted on the bank-projected states, with one stored code per training state. The correction directions C_q follow, as specified for each PDE. Sizes, cohorts, checkpoint hashes and recorded offline costs are in Appendix D.

5 EXPERIMENTAL SETUP AND BENCHMARK PROBLEMS

Problems and references. In two dimensions: Poisson on the square and on an L-shaped domain, heat, viscous Burgers and reflective waves; in three: Poisson, heat, Burgers and incompressible Navier–Stokes. Meshes and cohorts are in Table D.2, the sampled families in Table D.3 and the three-dimensional configurations in Table D.11. Burgers provides the correction-rank study.

Accuracy. We report worst relative L^2 error over the stated cases and requested times, in percent. Poisson uses the steady reference norm and heat the current reference norm; Burgers and Navier–Stokes use the initial-field norm over evolved times; waves use the energy-state error over displacement and velocity. Same-grid error measures reduction error relative to the converged discrete solution. Error against a refined reference also includes discretisation error; the two are never interchanged. Settings for a held-out (final) cohort are fixed before its cases are accessed; all other cohorts are development cohorts.

Timing and speedup. Every accuracy–runtime pair comes from the same solver invocation, in double precision, with GPU burn-in, synchronised timing and medians of retained repetitions (protocol: Appendix D.1). GPU query time covers initialisation, the solve or rollout and the requested device outputs; complete-query time adds host transfers and is labelled separately. Training and compilation are offline costs. Speedup is $S = T_{\text{FOM}}/T_{\text{method}}$ with both times from the same allocation; each table names the FOM algorithm and shows its error. Solves that miss their stopping rule are reported as such and do not enter a speedup.

Baselines. Linear problems are compared with conjugate gradients (CG), including the L-shaped domain and the CG solve inside each Crank–Nicolson (CN) heat step. Burgers is compared with Newton–BiCGStab and Navier–Stokes with its CNAB2 time integrator. The named FOM of a row is that solver at its fastest tested setting that is at least as accurate as the accurate NM-ROM setting.

6 NUMERICAL EXPERIMENTS AND RESULTS

Table 1 reports the headline comparison against the named iterative solvers; the subsections that follow analyse the settings behind it and which knob to turn.

Table 1: NM-ROM against the named full-order solver at every measured problem and mesh. **Both NM-ROM columns of a row come from one frozen model:** *fast* is the uncorrected setting ($q = 0$) and *accurate* the selected accurate rank (Table D.4); no network is retrained between them. Error is the worst relative L^2 error over the cohort (%). Speedup is the FOM’s median time divided by the NM-ROM’s in the same GPU allocation, **bold** where the NM-ROM is faster. The FOM is the named iterative solver at its fastest tested setting that is at least as accurate as the accurate setting. For the 4096² Burgers fast column, the numerator is instead the time for Newton–BiCGStab, tol 10^{-3} , $\Delta t=0.01$, the fastest tested setting at least as accurate as the fast setting. The FOM column reports the accurate setting’s comparator. Marker definitions, the heat evolved-time errors and the quadrature labels are in Appendix D.1. Dashes mark settings not measured. Times, settings and complete-query speedups: Table D.4.

		NM-ROM accurate		NM-ROM fast		Full-order model (FOM)	
Problem	Mesh	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Solver
<i>Two-dimensional</i>							
Poisson	256 ²	0.97	3.13 ×	3.16	3.33 ×	0.15	CG, rtol 10 ^{−2}
	1024 ²	0.96	13.9 ×	3.15	15.2 ×	0.072	CG, rtol 10 ^{−2}
	2048 ²	0.96	72.8 ×	3.15	74.3 ×	0.049	CG, rtol 10 ^{−2}
	4096 ²	0.96	116 ×	3.15	117 ×	0.45	CG, rtol 10 ^{−1}
Poisson, L-shape ^c	256 ²	2.13	3.84 ×	3.86	4.04 ×	0.64	CG, rtol 10 ^{−2}
	512 ²	2.12	6.07 ×	3.85	6.18 ×	0.38	CG, rtol 10 ^{−2}
	1024 ²	2.20	8.38 ×	3.85	8.59 ×	1.05	CG, rtol 3×10 ^{−2}
	2048 ²	2.20	17.0 ×	3.85	17.1 ×	0.76	CG, rtol 3×10 ^{−2}
Heat ^r	64 ²	—	—	4.56	0.22×	1.59	CN–CG, rtol 10 ^{−2}
	256 ²	—	—	4.56	0.61×	0.93	CN–CG, rtol 10 ^{−2}
	1024 ²	—	—	4.56	4.85 ×	0.77	CN–CG, rtol 10 ^{−2}
Heat (wide bank) ^{t,f}	1024 ²	0.49	1.59 ×	1.36	1.57 ×	0.18	CN–CG, Δt=0.05, rtol 10 ^{−3}
	2048 ²	0.49	9.74 ×	1.36	9.59 ×	0.18	CN–CG, Δt=0.05, rtol 10 ^{−3}
	4096 ²	0.49	36.0 ×	1.36	35.4 ×	0.47	CN–CG, Δt=0.05, rtol 10 ^{−2}
Heat (wide bank, batched fit) ^{t,f}	1024 ²	0.49	13.9 ×	1.36	13.0 ×	0.18	CN–CG, Δt=0.05, rtol 10 ^{−3}
	2048 ²	0.49	60.0 ×	1.36	57.0 ×	0.18	CN–CG, Δt=0.05, rtol 10 ^{−3}
	4096 ²	0.49	103 ×	1.36	101 ×	0.47	CN–CG, Δt=0.05, rtol 10 ^{−2}
Burgers ^e	256 ²	0.51 ^s	0.043×	1.89	0.79×	0.049	Newton–BiCGStab, tol 10 ^{−3}
	512 ²	0.55 ^x	0.068×	2.14	1.31 ×	0.052	Newton–BiCGStab, tol 10 ^{−3}
	1024 ²	0.59 ^d	0.0037×	2.29	2.02 ×	0.034	Newton–BiCGStab, tol 10 ^{−4}
Burgers ^e	2048 ²	0.87 ^ℓ	0.99×	2.37	4.16 ×	0.049	Newton–BiCGStab, tol 3×10 ^{−3}
	4096 ²	0.60 ^w	4.87 ×	2.41	10.10 ×	0.050	Newton–BiCGStab, tol 3×10 ^{−3}
Burgers (held-out cases) ^{e,h}	2048 ²	1.31 ^ℓ	1.43 ×	9.03	5.63 ×	0.13	Newton–BiCGStab, tol 10 ^{−3}
	4096 ²	1.33 ^w	5.38 ×	9.03	10.10 ×	0.14	Newton–BiCGStab, tol 3×10 ^{−3}
Burgers, confirmed rule ^e	512 ²	0.56 ^v	0.091×	2.14	0.75×	0.050	Newton–BiCGStab, tol 3×10 ^{−3}
	1024 ²	0.59 ^v	0.32×	2.29	1.39 ×	0.048	Newton–BiCGStab, tol 3×10 ^{−3}
Burgers (earlier model) ^r	1024 ²	—	—	3.91	1.63 ×	2.39	Newton–BiCGStab, relaxed
<i>Three-dimensional</i>							
Poisson ^f	32 ³	0.26	0.94×	1.40	0.99×	0.16	CG, rtol 10 ^{−2}
	64 ³	0.26	1.33 ×	1.39	1.38 ×	0.11	CG, rtol 10 ^{−2}
Poisson (dev. sources)	128 ³	0.16	6.75 ×	0.55	6.98 ×	0.075	CG, rtol 10 ^{−2}
	256 ³	0.16	23.2 ×	0.55	23.7 ×	0.049	CG, rtol 10 ^{−2}
Heat (new bank) ^{t,f}	32 ³	0.11	0.057×	2.00	0.13×	0.078	CN–CG, Δt=0.025, rtol 10 ^{−4}
	64 ³	0.11	— ⁿ	2.00	0.21×	0.081	CN–CG, Δt=0.025, rtol 10 ^{−4}
	128 ³	0.11	0.28×	2.00	0.62×	0.082	CN–CG, Δt=0.025, rtol 10 ^{−4}
Heat (new bank, batched fit) ^{t,f}	32 ³	0.11	0.62×	2.00	0.57×	0.078	CN–CG, Δt=0.025, rtol 10 ^{−4}
	64 ³	0.11	0.94×	2.00	0.85×	0.081	CN–CG, Δt=0.025, rtol 10 ^{−4}
	128 ³	0.11	2.22 ×	2.00	2.10 ×	0.082	CN–CG, Δt=0.025, rtol 10 ^{−4}

6.1 ACCURACY AND SPEED AGAINST FULL-ORDER SOLVERS

Linear problems. On Poisson the accurate setting reaches 0.96 % error, 13.9× faster than CG at 1024² and 116× at 4096² with the same error; the corrections cost little speed because they are eliminated analytically (§3.2). On the L-shaped domain the accurate setting is 6.07× faster at 2.12 %. The earlier heat checkpoint crosses CG between 256² and 1024² (4.85× at 1024²). A wider heat bank, evaluated once on a sealed held-out cohort at every mesh, reaches 0.49 % over all times and is 36.0× faster than CN–CG at 4096² (103× with the batched fit). In three dimensions the held-out Poisson cohort gives 0.26 % at 1.33×. A new three-dimensional heat bank, evaluated once on a sealed cohort of 64 cases, meets the 1 % all-times target at every mesh (accurate 0.11 %); with

Table 2: Other nonlinear-manifold ROMs on the shared Burgers 2D family at 256^2 and 512^2 : worst and median evolved same-grid error over 32 validation cases (never used for any choice) and compiled-query memory, one allocation. Latent dimension is matched for the baselines and our fast setting; our accurate setting solves $k + q = 272$ unknowns. The shallow masked-autoencoder NM-LSPG of Kim et al. (2022) passed its reproduction gate on the third of three pre-registered attempts; its hyper-reduction was not reproduced, and its published encoder width does not fit our devices, so at 512^2 its width-capped encoder is training-time limited (Table D.7, which also states the data-matched row and the residual path: every row here, ours included, uses a dense residual, so no times are given). The convolutional-autoencoder NM-ROM (Lee and Carlberg, 2020) was not run.

Method	Unknowns	256^2			512^2		
		Worst (%)	Median (%)	MB	Worst (%)	Median (%)	MB
Kim et al. NM-LSPG	8	163.93	43.34	2480	175.71	55.48	9959
Kim et al. NM-LSPG	16	145.40	43.87	2567	174.99	58.56	10511
Kim et al. NM-LSPG	32	127.51	43.61	2742	252.33	56.72	10879
Kim et al. NM-LSPG, data-matched	16	120.33	38.71	2567	—	—	—
POD-LSPG	8	55.52	22.72	18	55.82	24.08	71
POD-LSPG	16	41.53	10.45	30	42.01	13.83	122
POD-LSPG	32	24.78	5.65	55	25.63	7.74	222
This work, fast ($q = 0, k = 16$)	16	6.79	0.66	625	7.72	0.69	2368
This work, accurate ($q = 256, k = 16$)	272	0.88	0.089	1769	1.05	0.080	6891

Crank–Nicolson stepping it stays slower than CN–CG ($0.28\times$ at 128^3) and only the batched fit overtakes it, at 128^3 ($2.22\times$) and not at 32^3 or 64^3 .

Burgers. The reduced solve costs the same at every mesh, so the fast setting overtakes Newton–BiCGStab by 1024^2 ($2.02\times$ faster at 2.29 % error). The accurate setting reaches 0.51 % at 256^2 and, on development cases, is slower than the FOM through 2048^2 . This includes the rules confirmed on independent re-draws at 512^2 and 1024^2 (rows marked v). The dense 1024^2 panel row’s stored $q = 128$ rule gives 1.04 % at $0.50\times$. At 4096^2 the accurate setting chosen on six development cases reaches 0.60 % and is $4.87\times$ faster than the fastest Newton setting at least as accurate. On 64 held-out cases the same setting gives 1.33 % at $5.38\times$. The lattice quadrature rule of those rows is validated on held-out reached states but not confirmed on independent re-draws: it passes five draws and fails the confirmation draw (Appendix D.1). The fast setting gives $10.10\times$ on both cohorts against the fastest tested Newton–BiCGStab setting at least as accurate as itself.

Resolution. Table 1 shows that the NM-ROM query time grows more slowly with the mesh than the iterative FOM’s: in every series the fast-setting speedup rises under the rule that selects each row’s comparator (the fastest tested setting at least as accurate as the accurate NM-ROM, or as the fast setting for the 4096^2 Burgers fast column), and the selected comparator may change with the mesh. The speedup also rises against one fixed tight setting where a series timed one (Table D.4).

On 32 validation Burgers cases at 256^2 and 512^2 (Table 2) our fast setting, at matched latent dimension, reaches worst errors of 6.79–7.72 %, against 120–252 % for the reproduced shallow masked-autoencoder NM-ROM and 25–56 % for POD-LSPG; the accurate setting, at 272 solved unknowns, reaches 0.88–1.05 %.

Where the method currently fails. Table 3 collects the three problems that miss their targets; they are excluded from Table 1. On three-dimensional Burgers the corrections cut the held-out worst error from 16.26 % to 4.40 %, but the FOM is faster and the refined-reference check fails, so only same-grid reduction error is established. On three-dimensional Navier–Stokes the corrections lower the worst error only from 20.34 % to 18.92 %; 31 of 32 held-out cases miss the 5 % target. A follow-up diagnosis attributes this to representation: the family is a translating vortex orbit that a fixed bank must rebuild, and a shift-tracking linear POD ROM, a different model, meets the target on a fresh held-out cohort (4.80 % worst, 0 of 32 over) at $0.47\times$ CNAB2. Four neural operators trained at 32^3 on the same data and wall budget beat the NM-ROM on both of these problems, and are behind it on Poisson 3D; none of them transfers to 64^3 (Appendix C). On reflective waves the fast setting beats CG, but its energy-state error, which includes velocity, is 11.34 %; corrections bring it to

Table 3: Problems on which the method misses its target: Navier–Stokes 5 % per held-out case; wave, an energy-state error within that of CG; Burgers 3D, faster than the FOM at its accuracy (no numeric error target was recorded). Error is the worst same-grid relative L^2 error (%) over evolved times; the wave error is the energy-state error over displacement and velocity. Speedup is against the named FOM in the same allocation.

Problem	Setting	Err. (%)	FOM err. (%)	Speedup	FOM
Burgers 3D, 32^3 (final)	$q = 0$	16.26	2.31	$0.038\times$	Newton–BiCGStab
	$q = 192$	4.40	2.31	$0.023\times$	Newton–BiCGStab
Navier–Stokes 3D, 32^3 (final)	$q = 0$	20.34	2.47	$0.0025\times$	CNAB2
	$q = 256$	18.92	2.47	$0.0007\times$	CNAB2
Wave 2D, 1024^2 (dev.)	$q = 0$	11.34	4.29	$3.03\times$	midpoint–CG
	$q = 32$	5.12	4.29	$0.23\times$	midpoint–CG

Table 4: Correction-rank tunability at 4096^2 . Each problem block is one frozen model; every speedup is the median GPU query time of *one* named full-order setting (the *FOM* row, the fastest at least as accurate as the block’s most accurate setting, in the same allocation) divided by the setting’s (ms). Error is the worst same-grid relative L^2 error (%), over evolved times for Burgers and heat. Burgers development (6 cases) and held-out (64 cases, never used for selection) share settings; heat uses the sealed held-out cohort, Poisson development sources. On Burgers q and M change together ($M \approx 4(k + q)$, $M = 544$ an alternative at $q = 256$; fixed- M ladder: Table D.8), $q > 0$ with the lattice rule marked ^w in Table 1. Each block uses one FOM time for all speedups, so its $q = 0$ ratio differs from Table 1’s fast column. Heat and Poisson vary q only. * looser stopping tolerance; [†] quadrature rule failed its held-out check.

Setting	Development cohort			Held-out cohort		
	Err. (%)	ms	Speedup	Err. (%)	ms	Speedup
<i>Burgers 2D (6 development / 64 held-out cases)</i>						
$q = 0, M = 64$	2.41	40.7	$12.9\times$	9.03	40.6	$13.2\times$
$q = 128, M = 576$	1.09	77.4	$6.77\times$	2.43	74.6	$7.21\times$
$q = 256, M = 544$	0.88	112	$4.67\times$	2.93	105	$5.11\times$
$q = 256, M = 1088$	0.60	127	$4.12\times$	1.33	113	$4.77\times$
$q = 256, M = 1088$, looser tol.*	0.60	108	$4.87\times$	1.33	100	$5.38\times$
$q = 256, M = 2176^{\dagger}$	0.46	132	$3.96\times$	1.13	126	$4.25\times$
FOM: Newton–BiCGStab, tol. 3×10^{-3}	0.050	524	$1\times$	0.14	537	$1\times$
<i>Heat 2D, wide bank (16 sealed held-out cases)</i>						
$q = 0$	—	—	—	0.84	32.2	$146\times$
$q = 32$	—	—	—	0.22	31.7	$149\times$
$q = 96$	—	—	—	0.082	24.4	$193\times$
FOM: CN–CG, $\Delta t = 0.025$, rtol 10^{-6}	—	—	—	0.045	4713	$1\times$
<i>Poisson 2D (12 development sources)</i>						
$q = 0$	3.15	16.7	$117\times$	—	—	—
$q = 64$	2.08	16.9	$116\times$	—	—	—
$q = 128$	1.55	16.9	$116\times$	—	—	—
$q = 256$	0.96	16.9	$116\times$	—	—	—
FOM: CG, rtol 10^{-1}	0.45	1955	$1\times$	—	—	—

5.12 %, still above CG’s, at a cost that removes the speedup. The three-dimensional solves evaluate the residual densely.

Settings. Moving between the two settings of a row never requires retraining, only new deployment arguments: this is what we mean by deployment-time tunability (times and settings in Table D.4; fixed- M evidence in Table D.8).

6.2 WHICH KNOB TO TURN

Correction rank is the accuracy control (with a smaller fixed test space the same ranks give a $1.22\times$ error reduction from $q = 0$ to 256, so the test count is held fixed along the ladder). Quadrature and the stopping tolerance are cost controls: they lower the runtime at little or no change in error. The iteration cap is a safeguard, not a control; a truncated solve fails (Appendix Tables D.9 and D.10).

On Burgers at 4096^2 , changing the correction rank trades accuracy for cost. Table 4 compares every setting with one shared Newton–BiCGStab setting ($4.12\text{--}12.9\times$ on development cases). Table 1’s fast column is a different comparison: $10.10\times$ on both cohorts against the Newton–BiCGStab comparator selected for the fast setting. On linear problems the corrections are eliminated analytically, so accuracy improves at nearly constant cost.

The multi-seed sealed ladder is in Appendix E.

Limitations. (i) Speedups use the named iterative full-order solvers at the stated tolerances; direct and spectral solvers and coarser discretisations are outside this study’s scope. The named FOM is more accurate in every row. For heat at 4096^2 , the linear-bank baseline (0.13% in $5.1\text{--}8.0\text{ ms}$) is faster than every NM-ROM setting, as in three dimensions (0.070% at 128^3). (ii) A wider Burgers bank cuts the 256^2 projection floor from 0.513% to 0.183% on its selection cases. With confirmed quadrature, it is less accurate on 64 held-out cases (1.601% at $6.70\times$) than the model in Table 1. Its settings with unconfirmed quadrature are in Appendix D.1. The correction subspace and weak test space, rather than the bank, limit accuracy here. Development accuracy does not transfer to held-out cases (the unrestricted solve in the 512-function bank reaches only 1.70%). Poisson errors lie near the bank floor (0.742% square, 0.143% cube); the L-shape (2.20% against a floor of 0.77%) is limited by the head. (iii) The fixed-test-space rank study uses one checkpoint. The multi-seed and sealed study varies $M = 4(k + q)$, so it is not a replication. FOM comparisons do not isolate the nonlinear head’s contribution over the bank’s linear span. (iv) Quadrature validation covers finitely many reached states, not a global certificate; no rule is used in three dimensions. Reduced solves can converge to a wrong branch. Lower same-grid error does not remove discretisation error. (v) Unmarked cohorts are small development cohorts; exceptions are marked final or held-out. Timings report medians without dispersion.

7 CONCLUSION AND FUTURE WORK

We built an NM-ROM whose distinguishing property is a deployment-time accuracy–cost choice from one trained model, through nested corrections that change its capacity after training; it keeps its accuracy under mesh refinement on two-dimensional Poisson, heat and Burgers, so its speedup over the named iterative solvers grows with resolution: at 4096^2 its accurate setting reaches 0.96% at $116\times$ on Poisson, 0.49% at $36.0\times$ on held-out heat and 1.33% at $5.38\times$ on held-out Burgers, far more accurate than a reproduced NM-ROM and POD-LSPG; its 4096^2 rule is not confirmed on re-draws, and the confirmed variant is slower than the FOM. The named solvers remain more accurate; three-dimensional heat meets its accuracy target but is faster than CN–CG only with a batched fit at 128^3 ; and three-dimensional Burgers, Navier–Stokes and the full wave state miss their targets (Table 3); future work is a better three-dimensional bank, a cheaper high-rank solve and quadrature in three dimensions.

REPRODUCIBILITY STATEMENT

Every table and prose number is generated from hash-pinned run records identifying source, checkpoint, settings, cohort, allocation and audit (Appendix D); failed settings are kept.

AI USE STATEMENT

Language-model assistants supported method and experiment development, implementation, analysis, auditing and drafting. Recorded solver runs produced the numbers and the generators reproduce them. The authors are responsible for the methods, results and text.

REFERENCES

J. Barnett and C. Farhat. Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction. *Journal of Computational Physics*, 464:111348, 2022.

- P. Benner, S. Gugercin, and K. Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.
- J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han. Once-for-All: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020.
- K. Carlberg. Adaptive h -refinement for reduced-order models. *International Journal for Numerical Methods in Engineering*, 102(5):1192–1210, 2015.
- K. Carlberg, C. Bou-Mosleh, and C. Farhat. Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86(2):155–181, 2011.
- A. Cohen and R. DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 24:1–159, 2015.
- R. Geelen, S. Wright, and K. Willcox. Operator inference for non-intrusive model reduction with quadratic manifolds. *Computer Methods in Applied Mechanics and Engineering*, 403:115717, 2022.
- G. H. Golub and V. Pereyra. The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate. *SIAM Journal on Numerical Analysis*, 10(2):413–432, 1973.
- C. Greif and K. Urban. Decay of the Kolmogorov N -width for wave problems. *Applied Mathematics Letters*, 96:216–222, 2019.
- J. A. Hernández, M. A. Caicedo, and A. Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.
- M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.
- Y. Kim, Y. Choi, D. Widemann, and T. Zohdi. A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder. *Journal of Computational Physics*, 451:110841, 2022.
- M. Kim, T. Wen, K. Lee, and Y. Choi. Physics-informed reduced order model with conditional neural fields. *arXiv preprint arXiv:2412.05233*, 2024.
- N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: learning maps between function spaces. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
- K. Lee and K. T. Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- D. W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.

- B. Peherstorfer and K. Willcox. Online adaptive model reduction for nonlinear systems via low-rank updates. *SIAM Journal on Scientific Computing*, 37(4):A2123–A2150, 2015.
- O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.
- L. Sirovich. Turbulence and the dynamics of coherent structures, Parts I–III. *Quarterly of Applied Mathematics*, 45(3):561–590, 1987.
- R. Ștefănescu and A. Sandu. Efficient approximation of sparse Jacobians for time-implicit reduced order models. *arXiv preprint arXiv:1402.2018*, 2014.
- M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, 2022.
- M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems*, 33, 2020.
- J. Weder, P. Schwerdtner, and B. Peherstorfer. Online-efficient model reduction on quadratic manifolds with Neural Galerkin schemes. *arXiv preprint arXiv:2412.17695*, 2024.
- H. Wu, H. Luo, H. Wang, J. Wang, and M. Long. Transolver: A fast transformer solver for PDEs on general geometries. In *International Conference on Machine Learning*, 2024.
- M. Yano and A. T. Patera. An LP empirical quadrature procedure for reduced basis treatment of parametrized nonlinear PDEs. *Computer Methods in Applied Mechanics and Engineering*, 344:1104–1123, 2019.
- J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang. Slimmable neural networks. In *International Conference on Learning Representations*, 2019.

A METHOD DETAILS

This appendix writes the reduced problem (3) out for each PDE: what the residual is, how the time step enters, and what is solved at each query. Poisson, heat and Burgers below are fully discrete weak least-squares problems; the wave arm (§A.4) is an explicit second-order latent integration and is not an instance of (3). Formulas are for the scalar two-dimensional Dirichlet case; three-dimensional and periodic-vector scope is stated in Appendix D.1.

A.1 ELLIPTIC INSTANCE: POISSON

For $Au = f$ we take $\Lambda_\star = \Lambda$. Testing the residual and rescaling the rows leaves a small linear expression in the coefficients,

$$r_w(z, y) = \Lambda^{-1}P(Au - f) = B_0(h_\theta(z) + C_q y) - b_0, \quad B_0 = PG, \quad b_0 = \Lambda^{-1}Pf. \quad (4)$$

Because the retained tests are orthonormal and $Au^\star = f$, this residual is exactly the tested error $P(u - u^\star)$, so the minimised objective is the squared discrete L^2 error of the reduced state in the retained modes; this holds only for the modes P retains.

Analytically eliminated corrections. Because the residual is linear in y , the best y for a given z can be written down instead of iterated for. Let $B_0 C_q = QR$ be a thin QR factorisation. Q does not depend on z , so the inner minimisation over y has the closed form $\mathcal{R}y = Q^\top(b_0 - B_0 h_\theta(z))$ and the minimised value is

$$\min_y \|B_0(h_\theta(z) + C_q y) - b_0\| = \|B_\perp h_\theta(z) - b_\perp\|, \quad B_\perp = (I - QQ^\top)B_0, \quad b_\perp = (I - QQ^\top)b_0. \quad (5)$$

The LM iteration runs on z alone with the deflated operator, and y is recovered by one triangular solve. Because the projector is constant the elimination introduces no approximation and $B_\perp D h_\theta$ is the exact Jacobian of the eliminated residual; the only dropped derivative anywhere in the elliptic solve is the head curvature. At $q = R$ the deflated operator is zero, the latent problem is degenerate by construction, and the model is the linear least-squares solve $\min_y \|B_0 y - b_0\|$; the timed top rung is that QR solve, not an LM iteration on a zero Jacobian. We report the reduced stationarity of the deflated problem, the augmented stationarity of the full problem, the backward error of the triangular recovery and the numerical rank of $B_0 C_q$.

A.2 PARABOLIC INSTANCE: HEAT

The reduced heat step is one nonlinear least-squares problem per time step. The semi-discrete equation $\dot{u} = -\kappa Au$ is advanced with Crank–Nicolson, $(I + \frac{\Delta t \kappa}{2} A)u^{n+1} = (I - \frac{\Delta t \kappa}{2} A)u^n$, and the reduced step substitutes the manifold into the fully discrete equation *before* projecting. With $u^n = u(z_n)$, projecting with P and row-scaling by $\Lambda_\star = I + \frac{\Delta t \kappa}{2} \Lambda$ gives

$$r_{w,n}(z) = B_0 h_\theta(z) - D B_0 h_\theta(z_n), \quad D = \text{diag}\left(\frac{1 - \frac{\Delta t \kappa}{2} \lambda_i}{1 + \frac{\Delta t \kappa}{2} \lambda_i}\right), \quad (6)$$

and $z_{n+1} = \arg \min_z \|r_{w,n}(z)\|_2$, warm-started at z_n . Two points are structural. The right-hand side is built from the *decoded current state* at every step: carrying the quantity the previous step already made small freezes the recursion and reproduces a one-step solution to round-off. And z_{n+1} enters through h_θ , so the step is a nonlinear least-squares problem, not a linear solve. With corrections, $c_n = h_\theta(z_n) + C_q y_n$ replaces $h_\theta(z_n)$:

$$(z_{n+1}, y_{n+1}) = \arg \min_{z, y} \|B_0(h_\theta(z) + C_q y) - D B_0 c_n\|_2, \quad (7)$$

where y is eliminated as in (5) and the *complete* previous coefficients c_n , corrections included, are carried into the next step. At $q = R$ the head is redundant and the step is the linear least-squares recurrence

$$c_{n+1} = B_0^+ D B_0 c_n, \quad (8)$$

with B_0^+ the pseudo-inverse ($M \geq R$; the numerical rank of B_0 is checked) and c_0 fitted to the supplied initial field. This endpoint is the weak Crank–Nicolson propagation of the bank coefficients,

with no head and no iteration. It is the “top rung” of the heat ladder and the linear-bank baseline of the text. The deployed path factors the damped normal matrix by Cholesky; a variant assembles the Gram matrix $S = B_0^\top B_0$ offline and forms $H = Dh_\theta^\top SDh_\theta$ directly, which is exact and is reported as an algebraic ablation of the same step.

A.3 NONLINEAR ADVECTION: BURGERS

Burgers is the only problem whose tested residual stays nonlinear in the coefficients, so nothing is eliminated in closed form. The full-order problem is $u_t + u(u_x + u_y) = \nu \Delta u$ with the non-conservative sign-upwind stencil

$$N(u)_{ij} = u_{ij} (\delta_x u + \delta_y u)_{ij}, \quad (\delta_x u)_{ij} = N \cdot \begin{cases} u_{ij} - u_{i-1,j}, & u_{ij} > 0, \\ u_{i+1,j} - u_{ij}, & u_{ij} \leq 0, \end{cases} \quad (9)$$

δ_y analogous and switching on the same centre value, ghost zeros on all four walls, and backward Euler in time, $r_n(u) = u - u^n + \Delta t (N(u) - \nu \Delta_h u)$. Projecting and row scaling by $\Lambda_\star = I + \Delta t \nu \Lambda$ gives, with $c = h_\theta(z) + C_q y$,

$$r_{w,n} = (I + \Delta t \nu \Lambda)^{-1} [B_0 c - B_0 c_n + \Delta t (\hat{N}(c) + \nu \Lambda B_0 c)], \quad (10)$$

where the diffusion term is exact and the row scaling is the diagonal of the linearised implicit operator in the modal basis, the modal form of the Helmholtz preconditioner the full-order Newton solver uses. Only $\hat{N}(c) \approx PN(Gc)$ requires a choice: dense evaluation at every interior node, the sampled rule $\hat{N}(c) = \sum_{i \in \mathcal{S}} w_i P_{:,i} N_i(Gc)$ with the sign switch retained at the retained nodes, or the precomputed quadratic form $\frac{1}{2} c^\top Q c$ with Q the symmetrised tensor $T_{iab} = \sum_x P_{ix} G_{xa} (DG)_{xb}$, which is an identity only where $u > 0$ at every stencil node of every trial state the solver visits. The last is an algebraic route we verified in-job (residual and Jacobian parity below 10^{-11} relative on sign-satisfying states) and do not time in this paper. Time stepping uses a fixed substep count per output interval; each step is warm-started at the previous code, and the linear extrapolation is used instead when, and only when, its residual norm is smaller; the 4096² rows also try a quadratic predictor under the same rule. The runs above 1024² assemble the Jacobian analytically instead of by forward-mode products and clip the step; in the settings listed in Table D.4 they solve the normal system by Cholesky factorisation. In every Burgers arm above 1024² except the 2048² development accurate arm, the damping λ is carried from one step to the next rather than reset to λ_0 .

Block-damped step. With $J = [J_z \ J_y]$ the Jacobian of (10) in (z, y) , obtained in one forward-mode pass, each iteration solves

$$\left(J^\top J + \lambda D_z + \varepsilon D_y \right) \begin{bmatrix} \delta z \\ \delta y \end{bmatrix} = -J^\top r_{w,n}, \quad \|\delta z\| \leq \Delta, \quad (11)$$

where $D_z = \text{diag}(\text{diag}(J_z^\top J_z), 0)$ carries the Levenberg–Marquardt damping on the latent block only, $D_y = \text{diag}(0, I)$ with a fixed small ridge ε , and Δ is the $q = 0$ trust radius. The correction step is therefore an undamped Gauss–Newton step on the current linearisation, not throttled by a radius calibrated for the latent code. The step is accepted only if the residual decreases; otherwise λ grows as in §A.6. Table E.3 compares it with joint damping and with variable projection.

A.4 SECOND-ORDER INSTANCE: REFLECTIVE WAVES

For $u_{tt} = c^2 \Delta u$ the bank is mass-orthonormal and the R weak tests are the bank itself, so projection is Galerkin with stiffness $K = G^\top M_h A G$, M_h the diagonal mass matrix. Writing the coefficients as $a(z, y) = h_\theta(z) + C_q y$ and differentiating twice in time, $\ddot{a} = Dh_\theta \ddot{z} + h_\theta''(z)[\dot{z}, \dot{z}] + C_q \ddot{y}$, the projected equation $\ddot{a} = -c^2 K a$ becomes a linear least-squares problem for the joint acceleration,

$$\begin{bmatrix} Dh_\theta(z) & C_q \end{bmatrix} \begin{pmatrix} \ddot{z} \\ \ddot{y} \end{pmatrix} = -\left(c^2 K a + h_\theta''(z)[\dot{z}, \dot{z}] \right), \quad (12)$$

whose curvature term $h_\theta''[\dot{z}, \dot{z}]$ is a forward-over-forward derivative of the head. The first-order system in $(z, y, \dot{z}, \dot{y})$ is advanced with classical RK4 at a fixed step, one solve of (12) per stage; there is no

implicit residual minimisation and no stationarity exit. Initial coordinates and velocities are obtained from the supplied displacement and velocity. At $q = R$ the head is dropped and $\ddot{a} = -c^2 K a$ is propagated exactly through the eigendecomposition $K = V \Lambda V^\top$, $a(t) = V(\cos(\omega t) \hat{a}_0 + \omega^{-1} \sin(\omega t) \hat{b}_0)$, $\omega = c\sqrt{\Lambda}$. The iterative full-order comparator uses implicit midpoint with CG. The error reported in Table 3 is the energy-state error over displacement and velocity.

A.5 EMPIRICAL QUADRATURE: THE FIT AND THE COUNTS

A rule is judged by how well its weighted sum over m nodes reproduces the tested advection term of the full grid. That relative error, on one state, is

$$\rho(u) = \frac{\|\sum_{i \in \mathcal{S}} w_i P_{:,i} N_i(u) - PN(u)\|_2}{\|PN(u)\|_2}, \quad \rho_{\max} \leq 0.116 \text{ (primary bar)}, \quad \rho_{\max} \leq 0.06 \text{ (tight)}, \quad (13)$$

over the held-out reachable states described below. The primary bar is the held-out ρ of the original model’s $q = 0$ rule at the state that carries its largest first-interval error, measured in an earlier diagnostic study and fixed before any certification job ran; the tight bar was fixed before the EQ-ladder job of Table E.2. A candidate node set $\mathcal{C}$ is drawn once with a fixed seed. For each of n_{fit} stored codes $c^{(\ell)}$ we evaluate the decoded state, form the per-node contributions $N_i(Gc^{(\ell)})$ and the exact full-grid target $PN(Gc^{(\ell)})$, and solve

$$\min_{w \geq 0} \|\mathcal{G} w - \beta\|_2, \quad \mathcal{G}_{(\ell,i),c} = P_{i,c} N_c(Gc^{(\ell)}), \quad \beta_{(\ell,i)} = [PN(Gc^{(\ell)})]_i, \quad (14)$$

rows normalised by their Euclidean norms, support grown greedily by the Lawson–Hanson criterion with an exact non-negative refit after each addition and a hard cap of m nodes, padded to the requested count if the support saturates. Three counts are separated: the requested node count m , the achieved support, and the number of fit states n_{fit} . In the certification study the fit states are reachable states (states of the dense solver’s own converged per-step trajectory on fit trajectories), the held-out states are 512 reachable states per rung from certification trajectories disjoint from both the fit and the evaluation cases, and ρ of (13) is evaluated on those. A deployed rule is a stored triple: node indices, weights, and the cached $m \times 5 \times R$ bank block.

A.6 SOLVER CONSTANTS AND EXIT CODES

The representation ablations separate three error quantities: the *bank floor*, the projection error onto range G , which no head or solver can beat; the *best-found* error, the smallest any point of the augmented manifold attains, located by a multistart oracle; and the *solved* error, what the deployed iteration actually returns.

Write J for the Jacobian of r_w in (z, y) . Each Levenberg–Marquardt attempt solves $(H + \lambda \text{diag}(\text{diag } H)) \delta = -g$ with $H = J^\top J$ and $g = J^\top r_w$, and accepts δ only if the residual strictly decreases. The stationarity measure of §3.2 is scale free: it divides the gradient norm by the sizes of the Jacobian and the residual,

$$\eta(z, y) = \frac{\|J^\top r_w\|_2}{\|J\|_F \|r_w\|_2} \leq \eta_{\text{tol}}, \quad (15)$$

so one tolerance η_{tol} serves every mesh and every rank. The elliptic solve starts from the cached training code nearest the projected source; the heat and Burgers queries start from the nearest codes and fit (z, y) to the supplied initial field.

Damping starts at $\lambda_0 = 10^{-6}$ (Poisson, Burgers) or 10^{-4} (heat); $\lambda \leftarrow \max(\lambda/3, 10^{-12})$ on acceptance and $\min(10\lambda, \lambda_{\max})$ on rejection; the trust radius is taken from the spread of the training codes. Exit codes differ per problem and are reported per problem. Burgers: 4 stationarity, 1 residual, 2 tiny step, 3 rejected at $\lambda \geq 10^{14}$, 0 budget. Poisson: 6 stationarity, 2 relative residual, 1 stall, 3 damping limit, 5 non-finite initial value, 0 budget. Heat: 1 stationarity, 2 tiny step, 3 damping limit, 4 non-finite, 0 budget. The heat criterion divides by $\|J\|_F$ only, applied to a residual already normalised by the target norm; Poisson and Burgers use (15) directly. An initial fit that meets its stopping rule is accepted as the starting state of the time loop; this acceptance rule was fixed before the evaluation jobs ran, and complete exit flags remain in the source records.

B ARCHITECTURE AND ONLINE DATA FLOW

Figure B.1 shows the offline preparation and the online solve of every NM-ROM query in the paper.

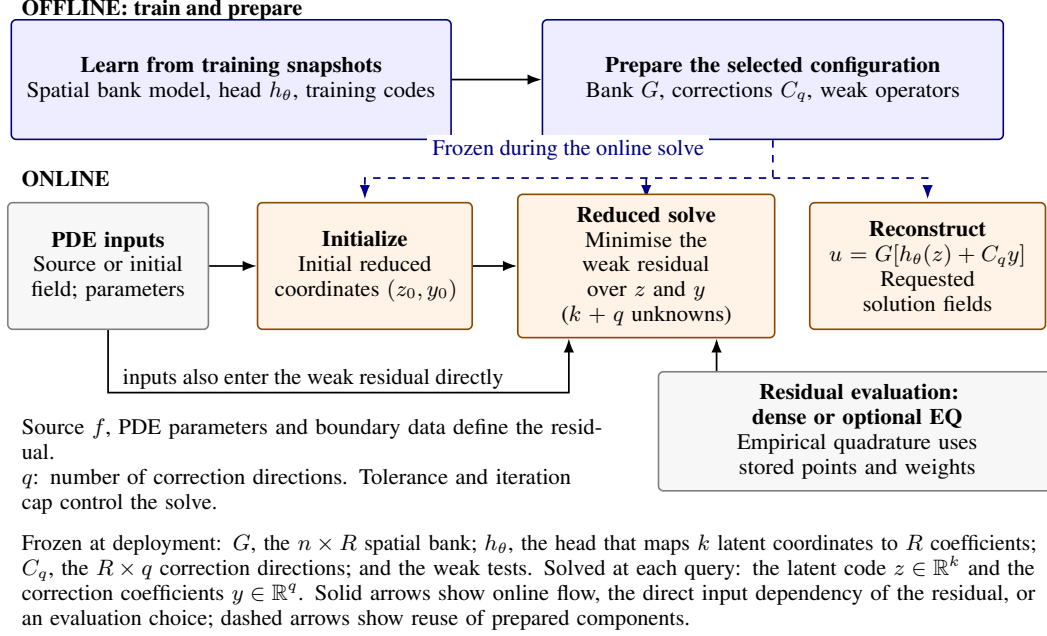


Figure B.1: NM-ROM from training to prediction, read left to right. Offline (blue), training fixes the spatial bank G , the head h_θ , the correction directions C_q and the weak tests; these never change again. Online (orange), the PDE inputs do two things: they set the starting reduced coordinates (z_0, y_0) , taken from stored training codes, and they enter the weak residual directly. The reduced solve then adjusts the $k + q$ unknowns (z, y) until that residual is small, and the reconstruction returns $u = G[h_\theta(z) + C_q y]$ at the requested times. Time-dependent problems repeat the solve once per step. Initialisation and the reduced solver are PDE-specific, and for linear PDEs the correction coefficients are eliminated analytically rather than iterated. Empirical quadrature (EQ) is an optional way to evaluate the residual, used for two-dimensional Burgers; the three-dimensional and wave solves evaluate it densely. Correction rank changes what the model can represent; EQ changes only how the residual is evaluated. Baselines are separate methods, not stages of this pipeline.

C NEURAL OPERATORS IN THREE DIMENSIONS

Four neural operators were trained on the same data, split and wall budget as the three-dimensional models of this paper and evaluated in the same allocation: an FNO (Li et al., 2021), a U-Net (Ronneberger et al., 2015), a DeepONet (Lu et al., 2021) and a Transolver (Wu et al., 2024). Tables C.1 and C.2 report worst same-grid error, median GPU query time and the speedup over the same full-order control used elsewhere in the row’s problem. They win where our three-dimensional models fail: on Burgers 3D the best of them, Transolver, reaches 2.02 % at $2.39 \times$ Newton–BiCGStab against 4.40 % for the NM-ROM, and the fastest is $3.94 \times$; on Navier–Stokes FNO reaches 0.47 % against 18.92 %. On Poisson 3D the corrected NM-ROM is the more accurate, and no operator is faster than CG (at most $0.998 \times$); on heat the best operator error is 0.32 %, with no CG measurement in that record and so no speedup. Five facts frame these numbers.

First, every operator here is trained and evaluated at 32^3 . The same trained operators transfer badly to 64^3 : worst errors of 17–220 % across the four families, recovered to 0.37–21.8 % only by training on the native grid and interpolating (for the FNO, also by physical padding). The top of that recovered range is again DeepONet. The fine-mesh rows of Table 1 are therefore NM-ROM against the named solvers alone; no operator was trained at those meshes.

Second, DeepONet is limited by the budget, not by its architecture: at these budgets it is the weakest arm everywhere (worst 26.9 %), and its recorded step budget and exit reason are in Table C.4.

Third, the operators produce the requested output times directly, in one pass. They are not autoregressive rollouts, so the set of output times is fixed when they are trained.

Fourth, each arm’s stop reason is recorded with its budget (Table C.4): every three-dimensional arm there stopped on its step budget rather than by early stopping, while the two-dimensional arms above train with an early-stopping patience. At equal wall time the float32 U-Net and Transolver completed more epochs than the float64 FNO, which favours them.

Fifth, an operator’s own evaluation resolution can trade accuracy for cost, as §2 says; the comparison here fixes one evaluation setting per trained operator.

Table C.1: Three-dimensional linear problems at 32^3 , 16 development cases: worst same-grid relative L^2 error, median GPU query time and speedup over the problem’s full-order control. Poisson uses the steady error and CG at $\text{rtol } 10^{-2}$ as the control; the heat record has no CG measurement, so no heat speedup is inferred.

Method	Poisson3D (development)			Heat3D (development)		
	Error (%)	GPU ms	S	Error (%)	GPU ms	S
NM-ROM, uncorrected	0.589	2.450	$1.06\times$	6.013	34.771	—
NM-ROM, corrected	0.161	2.632	$0.986\times$	1.379	114.343	—
POD	0.037	0.232	$11.2\times$	1.244	0.189	—
FNO	0.237	11.409	$0.227\times$	0.395	10.638	—
U-Net	0.193	4.099	$0.633\times$	0.318	4.373	—
DeepONet	1.807	2.599	$0.998\times$	21.769	3.889	—
Transolver	0.530	4.135	$0.627\times$	1.483	6.282	—
FOM	0.140	2.594	$1\times$	—	—	—

Table C.2: Three-dimensional nonlinear problems at 32^3 : Burgers on its accepted final cohort (32 cases, worst evolved error) and Navier–Stokes on a development cohort. Speedups divide the named full-order solver of the same problem in the same allocation.

Method	Burgers3D (final)			NS3D (development)		
	Error (%)	GPU ms	S	Error (%)	GPU ms	S
NM-ROM, uncorrected	16.263	219.161	$0.038\times$	13.535	1253.205	$0.0025\times$
NM-ROM, corrected	4.399	364.677	$0.0229\times$	12.534	4622.722	$0.000677\times$
POD	2.561	310.989	$0.0268\times$	23.732	30.460	$0.103\times$
FNO	2.378	6.072	$1.37\times$	0.470	8.903	$0.352\times$
U-Net	2.451	2.115	$3.94\times$	2.044	1.934	$1.62\times$
DeepONet	20.575	2.578	$3.23\times$	26.933	4.261	$0.735\times$
Transolver	2.017	3.489	$2.39\times$	2.691	3.361	$0.931\times$
FOM	2.312	8.337	$1\times$	0.628	3.130	$1\times$

Two dimensions at 256^2 . The Burgers operators were also timed against the NM-ROM, POD and the full-order solver in one allocation (job 4179247, 6 development cases, 5 retained repetitions each; Table C.3). The fast operators are quicker than the NM-ROM and than the full-order setting the rule matches to them (3 of 9 operator arms beat it, up to $1.48\times$), and an order of magnitude less accurate: 4.46–7.42 % worst evolved error, against 0.51 % for the accurate NM-ROM setting and 1.89 % for the fast one. Every operator error exceeds the mesh’s own discretisation error, 4.027 %: at this resolution they do not resolve the discrete solution they were trained on. No NM-ROM or POD arm here is faster than its comparator either; the accurate setting meets the campaign’s $\leq 1\%$ accuracy bar and misses its $\geq 5\times$ speed bar ($0.0659\times$), which is what Table 1 already shows for accurate Burgers below 4096^2 .

Two asymmetries are declared and left in place, both favouring the operators: their complete-query scope is not charged the upload of the input field, and the fields their errors are scored on come from an extra, untimed query. The FNO error reproduces the earlier panel case by case (difference $0.000e + 00$), and no timing was ever divided across jobs. This lane also could not reproduce an

earlier validation-selection pathology: here the selected arms are the better ones. The two cohorts are different splits with no case in common, so that is cohort-specific and refutes nothing.

Table C.3: Burgers 2D at 256^2 in one allocation (job 4179247): worst evolved and all-times same-grid error, median GPU query time, the full-order setting the rule matches to each row, and the speedup over it. Operator arms are the trained sizes of each family; the last three rows are the full-order settings the rule selects. Complete-query times add host transfer and are in the lane record.

Arm	Family	Evolved (%)	All times (%)	GPU ms	FOM by the rule	Speedup
NM-ROM, fast	NM-ROM	1.89	2.56	38.6	nt1e-3_dt01	0.50×
NM-ROM, accurate	NM-ROM	0.51	0.91	465	nt1e-3_dt005	0.07×
POD-LSPG, 256 modes	POD-LSPG	0.71	3.77	898	nt1e-3_dt005	0.03×
POD-LSPG, 512 modes	POD-LSPG	0.22	0.61	2770	nt1e-3_dt005	0.01×
FNO	FNO	7.42	7.42	7.3	nt1e-2_dt01	1.18×
U-Net, small	U-Net	5.14	5.14	9.4	nt1e-2_dt01	0.92×
U-Net, medium	U-Net	4.76	4.76	5.8	nt1e-2_dt01	1.48×
U-Net, large	U-Net	4.49	4.49	10.0	nt1e-2_dt01	0.86×
U-Net, refined	U-Net	4.55	4.55	5.9	nt1e-2_dt01	1.48×
Transolver, small	Transolver	4.90	4.90	11.0	nt1e-2_dt01	0.78×
Transolver, medium	Transolver	5.96	5.96	12.8	nt1e-2_dt01	0.67×
Transolver, large	Transolver	5.83	5.83	15.3	nt1e-2_dt01	0.57×
Transolver, refined	Transolver	4.46	4.46	11.1	nt1e-2_dt01	0.78×
nt1e-2_dt01	FOM	3.20	3.20	8.7	—	1×
nt1e-3_dt01	FOM	1.52	1.52	19.4	—	1×
nt1e-3_dt005	FOM	0.049	0.049	30.7	—	1×

Table C.4: Recorded training budgets of the neural operators: parameters, step budget, steps completed, the step selected by validation, batch, learning rate, seed and exit reason. Every arm exits on its step budget.

PDE	Operator	Parameters	Budget	Done	Selected	Batch	LR	Seed	Exit
Burgers 3D	fno	1771431	20000	20000	16100	2	0.001	920310	steps
Burgers 3D	unet	89431	20000	20000	17500	2	0.001	920311	steps
Burgers 3D	deeponet	1861959	30000	30000	30000	2	0.001	920312	steps
Burgers 3D	transolver	564392	30000	30000	29400	2	0.001	920313	steps
Heat 3D	fno_w16_m6	1771349	20000	20000	19500	4	0.001	920321	steps
Heat 3D	unet_w8	89197	20000	20000	20000	4	0.001	920322	steps
Heat 3D	deeponet_r128_w16	857749	20000	20000	20000	4	0.001	920323	steps
Heat 3D	transolver_w48_s32	562840	20000	20000	20000	4	0.001	920324	steps
Poisson 3D	fno_w24_m8	9440953	30000	30000	26750	2	0.0005	920450	steps
Poisson 3D	unet_w16	354577	30000	30000	28750	2	0.0005	920451	steps
Poisson 3D	deeponet_r128_w16	791697	30000	30000	29000	2	0.001	920452	steps
Poisson 3D	transolver_w48_s32	561272	30000	30000	29750	2	0.001	920453	steps
Navier–Stokes 3D	deeponet	6790319	32000	32000	23200	2	0.001	202609208	steps
Navier–Stokes 3D	transolver	569064	32000	32000	32000	2	0.001	202609209	steps
Navier–Stokes 3D	fno	4196511	8000	8000	7100	2	0.001	202609205	steps
Navier–Stokes 3D	unet	89287	8000	8000	7900	2	0.001	202609206	steps

D EXPERIMENTAL CONFIGURATION AND REPRODUCIBILITY

Tables D.2 and D.3 identify the two-dimensional studies and Table D.11 the three-dimensional ones. Within each study, training precedes evaluation and the selected bank, head and correction directions remain frozen. Deployment changes reduced coordinates, not network weights. Recorded training configurations and available offline costs are retained with the source evidence; unrecorded training costs are not inferred. The wide heat bank of Table 1 is a random-Fourier-feature coordinate network (32 features, scale 1.5, width 256, $R = 128$) trained for 80000 steps (learning rate 0.001, 64 states $\times$ 2048 points per batch, gradient clip 1, exact coefficient refit every 10000 steps), then a head of width 256 with $k = 8$ for 80000 steps (learning rate 0.0005, 128 states per batch), on 512 training draws at 64^2 with 16 validation draws; its validation projection floor is 0.11 %.

Full-order and solver details. The Burgers FOM is Newton’s method with BiCGStab inner solves (at most 200 iterations, Jacobian–vector products by forward-mode differentiation) preconditioned by

Table D.1: Training configurations of the frozen checkpoints, transcribed from the committed configuration and code files of each study (paths and SHA256 in evidence/training-configs-2026-09-21).

Model	Stage	Network	Optimiser, learning rate	Steps	Batch	Training data
Burgers 2D ($k = 16, R = 512$)	bank	128 Fourier features (scale 4), width 1024, 2 layers	AdamW, wd 10^{-5} , warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	300000	all states $\times$ 4096 points	16384 states of 576 trajectories
	head	width 512, 2 layers	Adam, warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	200000	4096 states	131072 states of 4608 trajectories
Poisson 2D ($k = 32, R = 512$)	bank	64 Fourier features (scale 4), width R , 2 layers	Adam, four phases at 10^{-3} , 3×10^{-4} , 3×10^{-4} , 10^{-4}	100000, 40000, 30000, 30000	64 sources $\times$ 4096 points	2611 fit / 461 validation sources
Poisson 2D ($k = 16, R = 128$)	head	width 128, 2 layers	Adam, 10^{-3}	150000 (full batch)	all	as bank
	bank + head	64 Fourier features, g and h width 128, 2 layers	Adam; staged from an $R = 64$ model (bank, head, joint at 3×10^{-4} , 3×10^{-4} , 10^{-5}) as Poisson $k = 32$	40000, 30000, 20000 after widening	64 sources $\times$ 4096 points	512 training sources
L-shape ($k = 16, R = 512$)	bank / head	64 Fourier features (scale 4), 2 layers; head width 128, 2 layers	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head	100000, 40000, 30000, 30000; head 150000	64 sources $\times$ 4096 points	2611 fit / 461 validation sources
Poisson 3D ($k = 16, R = 128$)	bank / head	64 Fourier features (scale 1.5), widths 256; head width 256 + skip	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head	150000 / 100000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation fields
Heat 3D, earlier bank ($k = 32, R = 128$)	bank / head	32 Fourier features (scale 1.5); head width 256 + skip	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head; clip 1, refit every 10000	150000 / 150000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation trajectories

the exact inverse of the implicit Helmholtz operator $(I + \Delta t \nu \lambda)^{-1}$ (advection dropped); Newton stops when the residual norm falls below the nonlinear tolerance times the previous state's norm, or after 20 iterations, and always takes the full step (no line search or damping). Each named setting fixes Δt , the nonlinear and the linear tolerance. On the L-shape the vanishing factor is $4 \min(x, y, d_{\text{right}}, d_{\text{top}}, d_{\text{corner}})$ with the distances to the outer right and top edges and to the re-entrant corner, and the test space is the M lowest eigenvectors of the five-point operator on the masked grid, computed by shift-invert Lanczos and scaled as $P = \Lambda^{-1} \Phi^T$ ($M = 257$; $M = 1024$ for the $q = R$ solve). The quadratic predictor of the 4096^2 Burgers rows forms the linear extrapolation $2w_n - w_{n-1}$ and, from the third step on, the quadratic one $3w_n - 3w_{n-1} + w_{n-2}$, evaluates the residual norm of the current code and both candidates in one batched call and starts the solve from the smallest.

Three-dimensional heat, new bank. The bank of the Heat (new bank) rows ($R = 320$) was trained with a code-free variable-projection trainer that eliminates the coefficients exactly; its projection floor is 0.06 %. Its arms were fixed by a rule registered before the sealed cohort was opened. A second model ($R = 256, k = 16, q = 160$), chosen after speed results had been seen, reaches 0.47 % on the same sealed cohort at 128^3 , $0.94 \times$ (Crank–Nicolson) and $1.92 \times$ (batched fit) against the rule's CN–CG setting; it is not used in Table 1.

Table D.2: Problem specification. Cohort and reduced sizes are read from the run configurations where recorded. The Burgers sealed cohort has been opened and is reported in Table E.1; other rows describe their recorded development and validation cohorts.

PDE	equation, domain, boundary	meshes	time stepping	reduced sizes	reference	cohorts
Burgers 2D	$u_t + u(u_x + u_y) = \nu \Delta u$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	256^2 – 4096^2	$\Delta t = 0.005$, backward Euler, sign-upwind	$k = 16, R = 512$	refined 4096^2 , $\Delta t = 0.00015625$	6 development cases; 32 validation; 64 held-out at 2048^2 , 4096^2 ; sealed cohort opened once
Poisson 2D	$-\Delta u = f$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	256^2 – 4096^2	none (elliptic)	$k = 16, R = 128$ (original); $k = 32, R = 512$	exact discrete solution; 2048^2 refinement	12 development sources
Heat 2D	$u_t = \kappa \Delta u$, $(0, 1)^2$	64^2 – 1024^2	Crank–Nicolson	$k = 8, R = 32$	refined-grid reference ($1024^2/2048^2$ pair); error includes discretisation	12 development cases, 3 repetitions; measured in job 3529772; checkpoint lineage: earlier cell, job 3511417
Heat 2D, wide bank	$u_t = \kappa \Delta u$, $(0, 1)^2$	1024^2 – 4096^2	Crank–Nicolson; batched exact-propagator fit	$k = 8, R = 128$	exact same-grid propagation	12 development cases; 16 sealed held-out cases, opened once
Wave 2D (reflective)	$u_{tt} = c^2 \Delta u$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	64^2 , 1024^2 , 256^2	RK4 on the manifold; exact modal propagation for the bank	$k = 32, R = 64$	exact modal solution	8 development cases
Poisson, L-shape	$-\Delta u = f$, $(0, 1)^2 \setminus [\frac{1}{2}, 1)^2$	256^2 – 2048^2	none	$k \in \{16, 32\}$, $R \in \{256, 512, 514\}$	converged discrete solution	3072 / 256 / 32 sources (train / selection / development)

D.1 HEADLINE SETTINGS, TIMES AND TIMING PROTOCOL

Table 1 markers. f held-out final cohort (all others development); r error against a refined reference, which includes discretisation error (all others same-grid); t all output times including $t = 0$; e evolved output times only; c complete-query time with host transfers (all others GPU query; heat was measured in GPU query only). Heat (wide bank) is a separately trained frozen model with a 128-function bank ($q = 32$ accurate); over evolved times its accurate error is 0.22 % (0.21 % batched; the batched fit:

Table D.3: Sampled problem families, transcribed from the generator sources named in the last column (code constants, not run outputs).

PDE	equation and boundary	sampled family (transcribed from the generator source)	source
Burgers 2D	$u_t + u(u_x + u_y) = \nu \Delta u$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	$u_0 = a \exp(- x - c ^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim U(0.05, 0.20)$, $a \sim U(0.5, 2)$; $\nu \sim$ $\log U(0.01, 0.1)$; outputs $t \in \{0.05, \dots, 0.25\}$	experiments/ mr-burgers2d/ engines.py: params_draw
Poisson 2D	$-\Delta u = f$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	$f = a \exp(- x - c ^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim$ $\log U(0.02, 0.1)$, $a \sim U(0.5, 2)$	multistage- precision/ms_ parametric.py: sample_params
Poisson, L- shape Heat 2D	same source family on $(0, 1)^2 \setminus [\frac{1}{2}, 1)^2$ $u_t = \kappa \Delta u$ on $(0, 1)^2$, $\kappa = 0.02$ fixed	as above; sources centred in the removed quadrant rejected polynomial-boundary Gaussian family (sin- gle_bc_poly_gaussian_v1); Crank-Nicolson $\Delta t = 0.025$	experiments/ lshape checkpoint_expanded_ seed790715 (lineage: linear-bank cell, job 3511417); printed CG measurements: job 3529772
Wave 2D (re- flective)	$u_{tt} = c^2 \Delta u$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	compact bump $\times$ Gaussian: half-widths $s_i \sim U(0.36, 0.42)$, centre $c_i \sim$ $U(s_i + 0.025, 1 - s_i - 0.025)$, amplitude $\sim U(0.7, 1.3)$, $\sigma_i \sim U(0.12, 0.16)$, advective velocity $v_i \sim$ $U(-0.5, 0.5)$ (zero every fourth case); speed $c \sim U(0.85, 1.15)$	experiments/ multiresolution- wave/audit_ dynamics.py: parameter_rows

Table D.4); Heat (new bank) ($q = 288$ accurate; Table D.11): at most 0.086 % (0.030 % batched) over evolved times; ⁿ 1 solve (one step of one case) missed its stationarity rule, so no speedup is given. Burgers accurate quadrature: ^s stored rule of job 3780164 ($m = 2560$; not the $q = 256$ rule of Table E.2), not confirmed on re-draws: it fails the confirmation draw at 256²; ^x its nodes with weights refit at 512² ($m = 2438$), one held-out draw, not re-drawn (with unrefit weights they pass 4 of 5 re-draws); ^l 63 $\times$ 63 lattice rule at 2048², thin margin: it clears the re-draw bar by only 4×10^{-4} ($\rho_{\max} = 0.1156$ vs. 0.116); ^w the same rule at 4096²: five held-out draws pass and the confirmation draw does not; ^v confirmed by the pre-registered re-draw procedure (1024²: thin margin, $\rho_{\max} = 0.1157$; none passes at 256²); ^d dense residual. The earlier Burgers model has one setting and may exit on a stall; its rule-selected FOM is the relaxed Newton setting ($10^{-2}/0.5$), and its tight-setting ratio is in Table D.4 with the other rows' tight ratios; the FOM setting can differ between meshes of a series. ^h 64 held-out cases never used for selection (not the sealed final cohort); at 2048² the development-chosen $M = 544$ setting was not run on them, so the $M = 1088$ setting is shown. Poisson (dev. sources) rows use development sources of a separate run and are not the held-out cohort.

Table D.4 gives the settings and measured times behind every row of Table 1. Every cost and error come from the same invocation. Timings use GPU warm-up and synchronisation, double precision and the highest matrix-multiplication precision; all repetitions, including outliers, enter the median, and no time is borrowed from another allocation. Requested full fields and initialisation are charged; host transfer is included only where the table says complete query. For the held-out Burgers rows, an independent restricted-grid recomputation of the errors disagreed with the full-grid value by more than 5 % on 5/384 (2048²) and 18/1152 (4096²) case-arm rows; per-arm cohort worsts agree within 1.2 % and the full-grid recomputation is exact. For Burgers at 256² no quadrature rule passed the pre-registered re-draw procedure (its pick failed the confirmation draw), so Table 1 has no confirmed-rule row there; a post-hoc follow-up rule with an exact first time step passes all 12 draws of two jobs and gives 0.520 % at $0.085 \times$ against the same Newton-BiCGStab rule. For Burgers at 4096², the lattice quadrature rule used in the accurate rows passes checks on five held-out trajectory samples but fails on the independent confirmation sample ($\rho_{\max} = 0.1173$ against the threshold 0.116). Restricting the same sample to time step 2 onward gives $\rho_{\max} = 0.0147$, locating the failure in the first steps. The rule is validated on held-out states but not confirmed on independent re-draws.

Using the exact residual at the first time step and the same quadrature rule thereafter passes all five checks and the confirmation check ($\rho_{\max} = 0.0513$). The error is unchanged at the reported precision. Median query times are 5308 ms on the six development cases and 6558 ms on the 64 held-out cases. The corresponding speedups against Newton-BiCGStab are $0.10 \times$ and $0.08 \times$. These queries cost $49.6 \times$ and $65.9 \times$ as much as queries using quadrature at every step in the same job (Table D.4). This variant is slower than the full-order solver on both cohorts. No confirmed form of this accurate

lattice rule is also faster. In the wider bank, the settings whose quadrature is not confirmed reach 0.393–0.458 % error at $1.98\text{--}2.21\times$ the speed of Newton–BiCGStab. The fast setting’s smaller rule passes the confirmation check and is faster than the full-order solver, at higher error.

For Burgers at 4096², copying the six double-precision output fields to the host adds about 246 ms to every arm. The Poisson and three-dimensional heat CG comparators are unpreconditioned; heat CG is warm-started from the previous time level. The Heat2D rows use checkpoint expanded_seed790715; Table D.2 separates that checkpoint’s lineage from the job that produced the printed measurements.

Table D.4: Supporting data for Table 1, including the full-order setting the rule selected for each row: the two settings of each frozen model, median times (ms), timing scope and evidence status (allocations: Appendix D). “EQ” and “dense” name the Burgers residual evaluation; “single” marks models measured at one setting. Where a run recorded both timing scopes, the speedups in the scope not used by Table 1 are listed; a series never mixes scopes. For heat that column gives the speedups against the tighter CN–CG setting (rtol 10^{-6}). The heat batched fit solves each output time independently against exactly propagated test moments, exploiting the linear, autonomous structure of the heat equation (eigenfunction tests). Rows from the paired-CG record list the one full-order setting that record retains, itself chosen by the same rule. * not in Table 1: a re-measurement of the row above it, a development-source run at a mesh whose held-out row is in Table 1, or the confirmed variant of the 4096² Burgers lattice rule with an exact residual at the first step. For the 4096² Burgers fast speedup in Table 1, the numerator is the time for Newton–BiCGStab, tol 10^{-3} , $\Delta t=0.01$, rather than the Newton time printed here.

Problem	Mesh	Accuracy	Fast	Accuracy ms	Fast ms	FOM setting	FOM ms	Timing	Other scope or FOM acc. / fast	Status
Poisson 2D	256 ²	$q=256$	$q=0$	7.12	6.70	CN, mol 10^{-2}	22.29	GPU query	—	development
Poisson 2D	1024 ²	$q=256$	$q=0$	4.32	3.94	CN, mol 10^{-2}	66.02	GPU query	—	development
Poisson 2D	2048 ²	$q=256$	$q=0$	5.47	5.15	CN, mol 10^{-2}	397.80	GPU query	$28.7\times / 28.8\times$ (complete query)	development
Poisson 2D*	4096 ²	$q=256$	$q=0$	16.92	16.74	CN, mol 10^{-1}	1954.80	GPU query	$32.7\times / 33.2\times$ (complete query)	development
Poisson 2D*	4096 ²	$q=256$	$q=0$	17.17	17.01	CN, mol 2×10^{-1}	1793.89	GPU query	$27.4\times / 27.6\times$ (complete query)	development
Poisson, L-shape 2D	256 ²	$q=64$	$q=0$	3.05	2.88	CN, mol 10^{-2}	11.64	complete query	—	development
Poisson, L-shape 2D	512 ²	$q=64$	$q=0$	4.80	4.72	CN, mol 10^{-2}	28.13	complete query	—	development
Poisson, L-shape 2D	1024 ²	$q=128$	$q=0$	5.81	5.67	CN, mol 3×10^{-2}	46.70	complete query	$13.7\times / 14.1\times$ (CPU query)	development
Poisson, L-shape 2D	2048 ²	$q=128$	$q=0$	18.75	18.66	CN, mol 3×10^{-2}	318.22	complete query	$29.8\times / 30.2\times$ (CPU query)	development
Heat 2D	64 ²	—	single	—	11.23	CN–CG, mol 10^{-2}	2.50	GPU query	—	development
Heat 2D	1024 ²	—	single	—	11.34	CN–CG, mol 10^{-2}	6.94	GPU query	—	development
Heat (wide bank) 2D	1024 ²	$q=32$	$q=0$	27.33	27.77	CN–CG, $\Delta t=0.05$, mol 10^{-3}	43.51	GPU query	$4.70\times / 4.82\times$ (vs. CN–CG mol 10^{-6})	development
Heat (wide bank) 2D	2048 ²	$q=32$	$q=0$	27.97	28.42	CN–CG, $\Delta t=0.05$, mol 10^{-3}	272.55	GPU query	$28.8\times / 28.3\times$ (vs. CN–CG mol 10^{-6})	sealed held-out
Heat (wide bank) 2D	4096 ²	$q=32$	$q=0$	31.68	32.19	CN–CG, $\Delta t=0.05$, mol 10^{-3}	1119.83	GPU query	$149\times / 146\times$ (vs. CN–CG mol 10^{-6})	sealed held-out
Heat (wide bank, batched fit) 2D	1024 ²	$q=32$	$q=0$	3.13	3.36	CN–CG, $\Delta t=0.05$, mol 10^{-3}	43.51	GPU query	$41.0\times / 38.3\times$ (vs. CN–CG mol 10^{-6})	sealed held-out
Heat (wide bank, batched fit) 2D	2048 ²	$q=32$	$q=0$	4.54	4.78	CN–CG, $\Delta t=0.05$, mol 10^{-3}	272.55	GPU query	$177\times / 169\times$ (vs. CN–CG mol 10^{-6})	sealed held-out
Heat (wide bank, batched fit) 2D	4096 ²	$q=32$	$q=0$	11.03	11.33	CN–CG, $\Delta t=0.05$, mol 10^{-3}	1119.83	GPU query	$428\times / 416\times$ (vs. CN–CG mol 10^{-6})	sealed held-out
Burgers 2D	256 ²	$q=256, M=1088, \text{EQ } m=2560$	$q=0, M=64, \text{EQ } m=1024$	746.02	40.36	Newton–BiCGStab, tol 10^{-3}	31.79	GPU query	—	development
Burgers 2D	512 ²	$q=256, M=1088, \text{EQ } m=2438$	$q=0, M=64, \text{EQ } m=922$	783.33	40.49	Newton–BiCGStab, tol 10^{-3}	52.92	GPU query	—	development
Burgers 2D	1024 ²	$q=256, M=1088, \text{dense}$	$q=0, M=64, \text{EQ } m=1024$	2303.85	40.27	Newton–BiCGStab, tol 10^{-3}	81.31	GPU query	—	development
Burgers 2D	2048 ²	$q=256, M=544, \text{EQ } m=3969$	$q=0, M=64, \text{EQ } m=1024$	124.15	29.48	Newton–BiCGStab, tol 3×10^{-3}	122.76	GPU query	$0.99\times / 2.01\times$ (complete query); $6.07\times / 25.5\times$ (vs. tight Newton)	development
Burgers 2D	4096 ²	$q=256, M=1088, \text{EQ } m=3969$	$q=0, M=64, \text{EQ } m=1024$	107.51	40.73	Newton–BiCGStab, tol 3×10^{-3}	523.85	GPU query	$2.18\times / 2.88\times$ (complete query); $30.4\times / 80.2\times$ (vs. tight Newton)	development
Burgers (held-out cases) 2D	2048 ²	$q=256, M=1088, \text{EQ } m=3969$	$q=0, M=64, \text{EQ } m=1024$	114.91	29.19	Newton–BiCGStab, tol 3×10^{-3}	164.43	GPU query	$1.28\times / 2.49\times$ (complete query); $6.48\times / 25.2\times$ (vs. tight Newton)	held-out
Burgers (held-out cases) 2D	4096 ²	$q=256, M=1088, \text{EQ } m=3969$	$q=0, M=64, \text{EQ } m=1024$	99.96	40.60	Newton–BiCGStab, tol 3×10^{-3}	557.39	GPU query	$2.28\times / 2.74\times$ (complete query); $32.3\times / 79.6\times$ (vs. tight Newton)	held-out
Burgers, exact first step 2D*	4096 ²	$q=256, M=1088, \text{EQ } m=3969, \text{first step exact}$	—	5308.00	—	Newton–BiCGStab, tol 3×10^{-3}	523.76	GPU query	—	confirmed rule
Burgers, exact first step 2D*	4096 ²	$q=256, M=1088, \text{EQ } m=3969, \text{first step exact}$	—	6557.77	—	Newton–BiCGStab, tol 3×10^{-3}	536.84	GPU query	—	confirmed rule
Burgers, confirmed rule 2D	512 ²	$q=256, M=1088, \text{EQ } m=3969, \text{first step exact}$	$q=0, M=64, \text{EQ } m=1024$	194.97	23.62	Newton–BiCGStab, tol 3×10^{-3}	17.74	GPU query	—	development
Burgers, confirmed rule 2D	1024 ²	$q=256, M=1088, \text{EQ } m=3969$	$q=0, M=64, \text{EQ } m=1024$	108.58	25.00	Newton–BiCGStab, tol 3×10^{-3}	34.60	GPU query	—	development
Burgers (earlier model) 2D	1024 ²	—	single	—	41.68	Newton–BiCGStab, relaxed	68.04	GPU query	—	development
Burgers (earlier model) 2D*	1024 ²	—	single	—	41.68	Newton–BiCGStab, right	667.75	GPU query	—	development
Poisson 3D	32 ³	$q=96$	$q=0$	2.63	2.49	CN, mol 10^{-2}	2.47	GPU query	—	accepted final
Poisson 3D	64 ³	$q=96$	$q=0$	3.38	3.22	CN, mol 10^{-2}	4.46	GPU query	—	accepted final
Poisson (dies, sources) 3D*	64 ³	$q=96$	$q=0$	1.43	1.38	CN, mol 10^{-2}	2.88	GPU query	$1.57\times / 1.56\times$ (complete query)	development
Poisson (dies, sources) 3D	128 ³	$q=96$	$q=0$	1.86	1.80	CN, mol 10^{-2}	12.55	GPU query	$2.70\times / 2.66\times$ (complete query)	development
Poisson (dies, sources) 3D*	128 ³	$q=96$	$q=0$	1.80	1.76	CN, mol 10^{-2}	12.40	GPU query	$2.65\times / 2.70\times$ (complete query)	development
Poisson (dies, sources) 3D	256 ³	$q=96$	$q=0$	6.08	5.95	CN, mol 10^{-2}	140.87	GPU query	$4.18\times / 4.21\times$ (complete query)	development
Heat (new bank) 3D	32 ³	$q=288$	$q=0$	38.16	25.41	CN–CG, $\Delta t=0.025$, mol 10^{-4}	3.33	GPU query	—	accepted final
Heat (new bank) 3D	64 ³	$q=288$	$q=0$	86.69	25.45	CN–CG, $\Delta t=0.025$, mol 10^{-4}	5.23	GPU query	—	accepted final
Heat (new bank) 3D	128 ³	$q=288$	$q=0$	62.03	28.25	CN–CG, $\Delta t=0.025$, mol 10^{-4}	17.44	GPU query	—	accepted final
Heat (new bank, batched fit) 3D	32 ³	$q=288$	$q=0$	5.36	5.88	CN–CG, $\Delta t=0.025$, mol 10^{-4}	3.33	GPU query	—	accepted final
Heat (new bank, batched fit) 3D	64 ³	$q=288$	$q=0$	5.54	6.14	CN–CG, $\Delta t=0.025$, mol 10^{-4}	5.23	GPU query	—	accepted final
Heat (new bank, batched fit) 3D	128 ³	$q=288$	$q=0$	7.84	8.30	CN–CG, $\Delta t=0.025$, mol 10^{-4}	17.44	GPU query	—	accepted final

Table D.5: Heat at high resolution, accuracy: one frozen model per block; worst same-grid relative L^2 error over all output times and over evolved times, accurate / fast setting. CN: reduced Crank–Nicolson steps; batched fit: each output time fitted independently to exactly propagated test moments (linear autonomous problems with eigenfunction tests only). The sealed 2D cohort was opened once and gives every wide-bank row of Table 1; the development row ran on another GPU (A100) and is shown for reference. Wide bank: $R=128, k=8$; earlier: the $R=32$ checkpoint of the Heat rows; 3D, earlier bank: the three-dimensional model that preceded the new bank of Table 1 (it misses the 1 % all-times target; 256³ is the first 16 final cases, not a mesh trend).

Model	Mesh	Cohort	Stepping	q acc./fast	Err. all times (%)	Err. evolved (%)
Wide bank	1024 ²	development (12)	CN	32 / 0	0.24 / 0.56	0.17 / 0.46
	1024 ²	sealed (16)	CN	32 / 0	0.49 / 1.36	0.22 / 0.84
	1024 ²	sealed (16)	batched fit	32 / 0	0.49 / 1.36	0.21 / 0.84
	2048 ²	sealed (16)	CN	32 / 0	0.49 / 1.36	0.22 / 0.84
2048 ²	sealed (16)	batched fit	32 / 0	0.49 / 1.36	0.21 / 0.84	
4096 ²	sealed (16)	CN	32 / 0	0.49 / 1.36	0.22 / 0.84	
4096 ²	sealed (16)	batched fit	32 / 0	0.49 / 1.36	0.21 / 0.84	
Earlier ($R=32$)	2048 ²	development (12)	CN	24 / 0	1.68 / 4.56	1.11 / 4.56
	4096 ²	development (12)	CN	24 / 0	1.68 / 4.56	1.11 / 4.56
3D, earlier bank	128 ³	final (64)	CN	96 / 0	1.93 / 3.18	0.75 / 1.54
	128 ³	final (64)	batched fit	96 / 0	1.93 / 3.18	0.72 / 1.36
	256 ³	final, 1st 16 (16)	CN	96 / 0	1.27 / 2.02	0.55 / 0.91
	256 ³	final, 1st 16 (16)	batched fit	96 / 0	1.27 / 2.02	0.50 / 0.86

Table D.6: Heat at high resolution, timing for the rows of Table D.5: median GPU query times (ms), the FOM chosen by the rule of Table 1 and its speedup, and the speedup against CN-CG at $\Delta t = 0.025$, $\text{rtol } 10^{-6}$.

Model	Mesh	Cohort	Stepping	ms acc./fast	FOM ms	CN-CG FOM	Speedup	vs. 10^{-6}
Wide bank	1024 ²	development (12)	CN	31.2 / 36.9	66.8	$\Delta t=0.05$, $\text{rtol } 10^{-3}$	2.14 / 1.81 $\times$	6.33 / 5.35 $\times$
	1024 ²	sealed (16)	CN	27.3 / 27.8	43.5	$\Delta t=0.05$, $\text{rtol } 10^{-3}$	1.59 / 1.57 $\times$	4.70 / 4.62 $\times$
	1024 ²	sealed (16)	batched fit	3.1 / 3.4	43.5	$\Delta t=0.05$, $\text{rtol } 10^{-3}$	13.9 / 13.0 $\times$	41.0 / 38.3 $\times$
	2048 ²	sealed (16)	CN	28.0 / 28.4	272.5	$\Delta t=0.05$, $\text{rtol } 10^{-3}$	9.74 / 9.59 $\times$	28.8 / 28.3 $\times$
	2048 ²	sealed (16)	batched fit	4.5 / 4.8	272.5	$\Delta t=0.05$, $\text{rtol } 10^{-3}$	60.0 / 57.0 $\times$	177 / 169 $\times$
	4096 ²	sealed (16)	CN	31.7 / 32.2	1139.8	$\Delta t=0.05$, $\text{rtol } 10^{-2}$	36.0 / 35.4 $\times$	149 / 146 $\times$
Earlier ($R=32$)	4096 ²	sealed (16)	batched fit	11.0 / 11.3	1139.8	$\Delta t=0.05$, $\text{rtol } 10^{-2}$	103 / 101 $\times$	428 / 416 $\times$
	2048 ²	development (12)	CN	15.9 / 8.9	173.1	$\Delta t=0.1$, $\text{rtol } 10^{-2}$	10.9 / 19.5 $\times$	51.1 / 91.4 $\times$
3D, earlier bank	4096 ²	development (12)	CN	18.0 / 10.8	1014.6	$\Delta t=0.1$, $\text{rtol } 10^{-2}$	56.5 / 93.7 $\times$	267 / 442 $\times$
	128 ³	final (64)	CN	35.1 / 16.5	7.0	$\Delta t=0.1$, $\text{rtol } 10^{-2}$	0.20 / 0.43 $\times$	0.83 / 1.78 $\times$
	128 ³	final (64)	batched fit	5.2 / 4.0	7.0	$\Delta t=0.1$, $\text{rtol } 10^{-2}$	1.36 / 1.77 $\times$	5.67 / 7.40 $\times$
	256 ³	final, 1st 16 (16)	CN	37.0 / 21.0	173.3	$\Delta t=0.05$, $\text{rtol } 10^{-4}$	4.69 / 8.27 $\times$	8.96 / 15.8 $\times$
	256 ³	final, 1st 16 (16)	batched fit	9.6 / 8.4	173.3	$\Delta t=0.05$, $\text{rtol } 10^{-4}$	18.1 / 20.5 $\times$	34.7 / 39.3 $\times$

Table D.7: Supporting data for Table 2: every row of that table with its residual path and median GPU query time from the same allocation, plus the exploratory hyper-reduced Kim et al. rows, whose hyper-reduction did not pass its reproduction gate. Our rows use the unoptimised dense reference path (no empirical quadrature, stopping tolerance 10^{-6} , $M = 4(k + q)$ tests); our optimised query is timed only in Table 1, in other jobs, and no ratio across jobs is formed. The shallow masked-autoencoder NM-LSPG of Kim et al. (2022) passed its reproduction gate, on their own 2D Burgers benchmark (their Sec. 6.2), on the third of three pre-registered attempts (sigmoid activation; median 1.45 % against a 1.5 % bar; published <1 %). Its published encoder width ($2n$) needs 136 GB to train at 256^2 and 2183 GB at 512^2 , beyond the 77 and 135 GB devices; at 512^2 its encoder, capped at width 4096, trained only 229–231 epochs in its 4800 s budget, so those errors partly reflect a training-time limit. The data-matched row is trained on our bank’s 576 trajectories.

Mesh	Method	Residual path	k	Worst (%)	Median (%)	GPU ms (same job)	MB
256 ²	Kim et al. NM-LSPG	dense	8	163.93	43.34	1301	2480
256 ²	Kim et al. NM-LSPG	dense	16	145.40	43.87	2384	2567
256 ²	Kim et al. NM-LSPG	dense	32	127.51	43.61	3846	2742
256 ²	Kim et al. NM-LSPG, data-matched	dense	16	120.33	38.71	3886	2567
256 ²	POD-LSPG	dense	8	55.52	22.72	138	18
256 ²	POD-LSPG	dense	16	41.53	10.45	259	30
256 ²	POD-LSPG	dense	32	24.78	5.65	519	55
256 ²	This work, fast ($q = 0$, $k = 16$)	dense (reference path)	16	6.79	0.66	262	625
256 ²	This work, accurate ($q = 256$, $k = 16$)	dense (reference path)	272	0.88	0.089	3746	1769
256 ²	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	8	115.60	51.78	65	2452
256 ²	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	16	202.87	52.51	126	2506
256 ²	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	32	122.68	41.98	475	2794
512 ²	Kim et al. NM-LSPG	dense	8	175.71	55.48	746	9959
512 ²	Kim et al. NM-LSPG	dense	16	174.99	58.56	1497	10511
512 ²	Kim et al. NM-LSPG	dense	32	252.33	56.72	2059	10879
512 ²	POD-LSPG	dense	8	55.82	24.08	314	71
512 ²	POD-LSPG	dense	16	42.01	13.83	617	122
512 ²	POD-LSPG	dense	32	25.63	7.74	383	222
512 ²	This work, fast ($q = 0$, $k = 16$)	dense (reference path)	16	7.72	0.69	404	2368
512 ²	This work, accurate ($q = 256$, $k = 16$)	dense (reference path)	272	1.05	0.080	6269	6891
512 ²	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	8	273.27	53.08	30	9813
512 ²	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	16	185.17	58.52	62	10099
512 ²	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	32	194.47	56.00	138	10569

Table D.8: Burgers 2D correction settings from one frozen model at 256^2 , test count fixed at $M = 1088$, dense residual, one allocation. Errors are worst evolved same-grid percentages on development cases; times are paired median GPU query times. All rows meet the stopping rule.

Correction rank q	Relative L^2 error (%)	GPU query time (ms)
0	1.2657	848.0
64	1.0593	1359.4
128	0.8711	1886.5
256	0.5194	4377.9

Table D.9: Which knob to turn, measured on Burgers 2D from frozen models. Rank: fixed- M ladder at 256^2 (Table D.8). EQ: dense time over EQ time at $q = 0$ in one allocation per mesh. Tolerance and cap: 32 validation cases (Table E.4), stopping tolerance $10^{-8} \rightarrow 10^{-3}$ and iteration cap 2. * Jobs 3789570 and 3789572; Table D.10 is another allocation.

Knob	Turn it to	Effect on error	Effect on cost
Correction rank q	$0 \rightarrow 256$	$2.44\times$ lower	$5.16\times$ higher
Empirical quadrature	dense $\rightarrow$ stored rule	Table D.10	$4.77\times$ lower at 256^2 , $38.53\times$ at 1024^{2*}
Stopping tolerance	looser	$6.712 \rightarrow 6.701$ %	35.6 % lower
Iteration cap	2 iterations	fails (90.3 %)	—

Table D.10: Dense and empirical-quadrature (EQ) residual evaluation on Burgers 2D at 256^2 . Error is worst evolved relative L^2 (%); time is median GPU milliseconds from one allocation (job 3780164, the EQ-ladder job of Table E.2; the 256^2 row of Table 1 is job 3789570 and Table D.8 is the fixed- M ladder job, each with its own measured error); the last column is dense time over EQ time. Dashes: no paired dense measurement. Rules at $q = 0, 16, 32$ pass every independent reconstruction; higher-rank rules pass some (Table E.2).

Correction rank q	Dense error (%)	Dense ms	EQ error (%)	EQ ms	Dense / EQ
0	1.8890	292.87	1.8891	59.07	$4.96\times$
16	—	—	1.4270	80.61	—
32	—	—	1.2493	97.70	—
64	1.0843	624.93	1.2275	120.63	$5.18\times$
128	0.8930	1190.49	0.8936	246.87	$4.82\times$
256	0.5194	3915.14	0.5389	722.21	$5.42\times$

Table D.11: Three-dimensional configurations, read from the run records. Domains are the unit cube with homogeneous Dirichlet data, except Navier–Stokes, which is periodic. Burgers and Navier–Stokes appear in Table 3.

Problem	Mesh	Cases	Correction ranks	Named FOM	Residual
Burgers 3D	32^3	32 final	0, 192	Newton–BiCGStab, $\Delta t = 0.01$	dense
Poisson 3D	$32^3, 64^3$	64 final	0, 32, 96 ($k = 16$)	CG, $\text{rtol } 10^{-2}$, no preconditioner	dense
Heat 3D (new bank)	$32^3, 64^3, 128^3$	64 sealed final (all times)	0, 288 ($k = 32, R = 320$)	CN–CG, setting per row by the rule; named $\Delta t = 0.025$, $\text{rtol } 10^{-6}$	exact (linear)
Navier–Stokes 3D	32^3 periodic	32 final	0, 32, 64, 128, 256 ($k = 64, R = 1536, M = 2048$)	CNAB2, $\Delta t = 0.01$	dense

Three-dimensional Poisson ($-\Delta u = f$, Gaussian sources), heat ($u_t = \kappa \Delta u$, Gaussian initial fields) and scalar Burgers ($u_t + u(u_x + u_y + u_z) = \nu \Delta u$, backward Euler, sign-upwind) are posed on the unit cube with zero Dirichlet walls and extend the operators of Appendix A across three axes. Navier–Stokes uses periodic vector fields, an incompressibility projection and the CNAB2 integrator; its FOM is not a CG solve. All four three-dimensional problems are evaluated on held-out final cohorts whose settings were frozen beforehand.

D.2 SOURCE RECORDS

The accompanying source package retains the full experiment archive, including failed configurations, retractions, training records, checkpoints, case-level fields, timing repetitions and independent audits. The PDF contains the evidence needed for its stated claims rather than every experiment in that archive. Table generators and frozen evidence identify the displayed values through these machine-readable manifests:

- `tables/provenance.json`: original two-dimensional studies.
- `tables/rewrite-provenance.json`: paired CG comparisons.
- `tables/burgers-iterative-provenance.json`: the earlier fine-grid Burgers tight and relaxed controls.
- `tables/headline-provenance.json`: every row of Tables 1, D.4, 3 and D.11 and every row of Table 4, with the commit and SHA256 of each source record.

Each manifest records source hashes; its corresponding evidence retains solver configuration, checkpoint identity and allocation metadata. Development selection and final evaluation are distinct. Solver exit flags document numerical stopping, not a proof of global optimality.

Allocations. Every ratio pairs times from one Slurm allocation. Tables 1 and D.4: Poisson 2D 3780692, 3783813, 4049279, 4051236, 4071227; Poisson, L-shape 2D 3784663, 3789568, 4057694; Heat 2D 3529772; Heat (wide bank) 2D 4060928; Heat (wide bank, batched fit) 2D 4060928; Burgers 2D 3789570, 3805065, 3789572, 4071616, 4079320; Burgers (held-out cases) 2D 4077566, 4079321; Burgers, exact first step 2D 4153483; Burgers, confirmed rule 2D 4140818, 4139290; Burgers (earlier model) 2D 3534502; Poisson 3D 4028642; Poisson (dev. sources) 3D 4051032, 4056288; Heat (new bank) 3D 4153878; Heat (new bank, batched fit) 3D 4153878. Table 3: Burgers 3D 4021709; Navier–Stokes 3D 4027788; Wave 2D 3780450. Table D.11: Burgers 3D 4021709; Poisson 3D 4028642; Heat 3D 4153878; Navier–Stokes 3D 4027788. Table D.5: 4051290, 4054869, 4060928, 4072943. Tables 2 and D.7: 256^2 4095408, 512^2 4107272 (reproduction gate: 4077574).

Training/evaluation separation (Burgers). The 4608 training trajectories of the Burgers checkpoint were regenerated from their seeds and compared with every evaluation cohort (the 32 validation cases of Table 2, the 64 held-out cases, the 6 development cases, the sealed cohort): no cohort shares a parameter vector, or any single parameter value, with the training set (check script and output in `evidence/burgers-train-eval-overlap-2026-09-21`).

E VALIDATION OF THE CORRECTION AND QUADRATURE STUDIES

A separate ladder with $M = 4(k + q)$, evaluated once on a sealed cohort, gives monotone error reduction for all 4 checkpoints (top-rank errors 0.59–0.68 %, Table E.1); 3 of 4 meet the pre-registered secondary criterion (settings spanning at least $2\times$ in error and in cost). Two stricter pre-registered checks fail: the original model’s sealed-to-development ratio at $q = 0$ (one sealed case converges to a wrong branch, 10.1120 %) and universal convergence (the second retrained seed at $q = 64$ (3 budget exits); Table E.1).

Reading the validation tables. Correction rank q is the number of added coefficient directions; M is the number of weak test modes and m the quadrature node count. A scheduled ladder changes M with q ; the main fixed-test-space study holds M constant. A sealed cohort is opened only after choices are frozen. Confirmed quadrature passes every independent reconstruction of the rule; marginal means some reconstructions fail. These statuses concern rule construction, not just the error of one selected trajectory. Same-grid error and error against a refined reference measure different quantities.

Table E.1: The sealed cohort (job 3804465, NVIDIA A100-PCIE-40GB), opened once after every choice was frozen. Top: per rung of the dense $M = 4(k + q)$ ladder, the original model’s sealed worst evolved error and device time, the seed mean $\pm$ sample standard deviation on the sealed and the development cohort, the pre-registered sealed-over-development ratio of seed means (raw and difficulty-normalised) and the converged count over the 4 checkpoints. Bottom: per-checkpoint verdicts. The original model’s $q = 0$ rung is a single sealed case converged to a wrong branch (gradient exit, zero budget exits); the pre-registered ratio criterion fails at $q = 0$ for the original model alone (5.35) and the convergence criterion fails on the second retrained seed at $q = 64$ (3 budget exits). Top-rank sealed errors per checkpoint: original 0.68 %, first retrained seed 0.61 %, second 0.59 %, third 0.66 %.

q	M	original model sealed %	ms	seeds sealed %	seeds development %	sealed / dev	normalised	converged
0	64	10.1120	375.1	2.8023 ± 0.6357	2.0002 ± 0.4986	1.40	1.41	4 of 4
16	128	1.4617	480.8	1.7305 ± 0.1297	1.5988 ± 0.0995	1.08	1.09	4 of 4
32	192	1.4458	540.5	1.5044 ± 0.1422	1.3727 ± 0.1314	1.10	1.14	4 of 4
64	320	1.3252	754.5	1.3528 ± 0.1099	1.2130 ± 0.0580	1.12	1.13	3 of 4
128	576	1.0964	1460.4	1.0461 ± 0.0619	0.9562 ± 0.0264	1.09	1.17	4 of 4
256	1088	0.6789	6725.5	0.6188 ± 0.0345	0.4987 ± 0.1164	1.24	1.20	4 of 4

checkpoint	job	monotone (evolved)	monotone (all-times)	every setting converged	error span	cost span	secondary criterion
original	3804465	yes	yes	yes	$14.89 \times$	$17.93 \times$	yes
first seed	3804465	yes	yes	yes	$3.52 \times$	$13.25 \times$	yes
second seed	3804465	yes	yes	no	$4.81 \times$	$13.17 \times$	no
third seed	3804465	yes	yes	yes	$5.22 \times$	$14.62 \times$	yes

Table E.2: The EQ ladder with the cheapest rule passing the primary bar in its draw per rung, timed in one allocation (job 3780164, A100 80 GB), with its same-job dense twins, and the construction status from the four-draw replication (job 3783811): confirmed = every re-draw passes, marginal = only the listed fraction does. Errors are as measured with these rules (SHA256-identified); they are not properties of the construction above $q = 32$. Ladder rows are read from the study’s final summary record.

q	m	fit states	$\rho_{\max}$ (this draw)	construction status	EQ evolved %	EQ all %	EQ ms	dense evolved %	dense ms	EQ/dense cost
0	1024	64	0.0153	confirmed (3 of 3 re-draws)	1.8891	2.5629	59.1	1.8890	292.9	0.202
16	1024	64	0.0935	confirmed (3 of 3 re-draws)	1.4270	2.4806	80.6	—	—	—
32	1024	42	0.0533	confirmed (2 of 2 re-draws)	1.2493	2.3534	97.7	—	—	—
64	1024	25	0.0531	marginal (2 of 6 draws pass)	1.2275	2.1489	120.6	1.0843	624.9	0.193
128	2048	64	0.0669	marginal (4 of 5 draws pass)	0.8936	1.8116	246.9	0.8930	1190.5	0.207
256	2048	64	0.1074	marginal (1 of 5 draws pass)	0.5389	0.9053	722.2	0.5194	3915.1	0.184

Table E.3: How the corrections are solved on Burgers, one job (job 3734098, NVIDIA A100 80GB PCIe): joint LM on (z, y) , block-damped, and plain variable projection at $q = 64$ and 128. Same error where all converge; the block-damped step is the one kept.

arm	M	worst all-times %	budget exits	all stationary	device ms
$q=64$, joint LM on (z, y)	320	2.1489	6	no	1119.7
$q=64$, block-damped	320	2.1489	0	yes	619.2
$q=64$, plain variable projection	320	2.1489	0	yes	9527.9
$q=128$, plain variable projection	256	1.8116	297	no	47566.9
$q=128$, block-damped	256	1.8116	0	yes	825.5

Table E.4: Solver-side controls on Burgers 2D, 32 validation cases, one allocation: stopping tolerance, EQ node count m and iteration cap, with full-order settings for scale. Error: worst over cases and output times of the L^2 error against the refined reference, relative to the initial-field norm, so the full-order rows show their discretisation error.

setting	worst % (32 validation)	device ms	early-stopped
EQ $m=256$, tol 10^{-6} , cap 180	7.245	51.0	0/96
EQ $m=512$, tol 10^{-8}	6.712	64.7	0/96
EQ $m=512$, tol 10^{-3}	6.701	41.6	0/96
EQ $m=512$, cap 2 (early-stopped)	90.324	28.4	96/96
FOM Newton 10^{-2} , $\Delta t=0.01$	6.807	9.9	0/96
FOM Newton 10^{-2} , $\Delta t=0.005$	35.357	17.4	0/96
FOM Newton 10^{-4} , $\Delta t=0.005$	6.171	22.4	0/96
FOM Newton 10^{-6} , $\Delta t=0.005$	6.172	88.3	0/96